Lecture 9: Synthetic Control Methods, Part 2

James Sears*
AFRE 891/991 FS 26
Michigan State University

*Parts of these slides are adapted from **"Causal Inference for the Brave and True"** by Matheus Facure Alves.

Table of Contents

1. Prologue

2. Synthetic Difference-in-Difference

3. Partially Pooled Synthetic Control

4. Synthetic Control Prediction Intervals

5. SCM Assumptions

6. Not Covered: PPSCM Application

Prologue

Prologue

Part 1

- Matching
- Canonical Synthetic Control

Part 2

- Synthetic Diff-in-Diff
  - Uniform Adoption
  - Staggered Adoption
- Partially Pooled Synthetic Control
- Synthetic Control Prediction Intervals

Prologue

Packages we'll use today:

```
if (!require("synthdid")) remotes::install_github("synth-
inference/synthdid")
if (!require("pacman")) install.packages("pacman")
pacman::p_load(fixest, gsynth, scpi, synthdid, tidyverse)
```

As well, let's load the event study data from **Sears et al. (2023)** we finished last lecture with (plus some covariates)

```
sah ← readRDS("data/sah_covar.rds")
```

Prologue

Last time we talked through the canonical **Synthetic Control Method (SCM)** for comparative case studies.

While it provides an avenue for overcoming violations of parallel trends, it is limited in several ways

1. Intended for a single treated aggregate unit ("comparative case studies")
2. Results primarily presented visually
3. Inference doesn't match our usual approaches

What would be *great* is if there were ways of combining the matching benefits of SCM with the identification and structure of our typical econometric approaches...

Prologue

Okay so I've already spoiled this: yes, there are methods that do this - and we're going to learn about them today.

1. **Synthetic Difference-in-Differences**: using a synthetic control unit as the counterfactual in a difference-in-differences
2. **Partially Pooled Synthetic Control:** first to extend to event study settings with dynamic treatment effects
3. **Cattaneo et al. (2025)** formalizes uncertainty for a range of estimands of interest

We refer to these approaches as **doubly-robust** estimators, in that you have two chances to be right:

1. In the matching design, or
2. In the regression specification

Synthetic Difference-in-Differences

Synthetic Difference-in-Differences

Synthetic Difference-in-Differences by Arkhangelsky et al. (2021) AER offers a solution to CIA violations in a typical difference-in-differences setting.

The gist:

1. Re-weight and match pre-exposure trends by constructing a synthetic counterfactual
   - Reduces reliance on parallel trends
2. Estimate a treatment effect in a standard TWFE regression

Synthetic Difference-in-Differences

More formally:

Suppose you have **balanced panel** with

- T time periods
- N units
  - First N_{co} units are never exposed to treatment (pure controls)
  - Next $N_{tr} = N - N_{co}$ units receive treatment after time T_{pre}
    - Can be block adoption or staggered adoption

You observe

- The time-varying outcome Y_{it},
- A binary treatment indicator W_{it},
- and (optionally) time-varying covariates X_{it}

i.e. same data requirements as a standard TWFE Diff-in-Diff approach

Synthetic Difference-in-Differences

The Concern: even after conditioning on observables, CIA/parallel trends doesn't hold.

The Solution: construct a synthetic control to use as the counterfactual in a Difference-in-Differences design.

Like with SCM, the goal is to find weights $\hat{\omega}^{sdid}$ so that the pre-treatment trends in the treated units' outcome align with the synthetic control's:

$$\underbrace{\sum_{i=1}^{N_{co}} \hat{\omega}^{sdid} Y_{it}}_{Synthetic\ Control} \approx \underbrace{\frac{1}{N_{tr}} \sum_{i=N_{co}+1}^{N} Y_{it}}_{Treated}$$

Synthetic Difference-in-Differences

Choosing Unit Weights: ensure average outcome for treated unit(s) is roughly parallel to the synthetic counterfactual (i.e. weighted average of control units)

Choose **unit weights** $(\hat{\omega}_0, \hat{\omega})$ that yield the arg min of

$$\underbrace{\sum_{t=1}^{T_{pre}} \left(\omega_0 + \sum_{i=1}^{N_{co}} \omega_i Y_{it} - \frac{1}{N_{tr}} \sum_{i=N_{co}+1}^{N} Y_{it} \right)^2}_{Sum\ Squared\ Error} + \underbrace{\zeta^2 T_{pre} ||\omega||_2^2}_{L_2\ Regularization\ Penalty}$$

Synthetic Difference-in-Differences

$$\underbrace{\sum_{t=1}^{T_{pre}}\left(\omega_0 + \sum_{i=1}^{N_{co}}\omega_i Y_{it} - \frac{1}{N_{tr}}\sum_{i=N_{co}+1}^{N} Y_{it}\right)^2}_{Sum\ Squared\ Error} + \underbrace{\zeta^2 T_{pre}||\omega||_2^2}_{L_2\ Regularization\ Penalty}$$

- Intercept term ω_0 allows synthetic control to match on pre-trends
  - Unit FE α_i in regression step get rid of level differences
- Penalty term helps non-zero weights be more distributed across control units
  - If $\zeta = 0$, weights are identical to Abadie, Diamond, and Hainmueller (2010) SCM weights for a single treated unit
  - ζ is *complicated*, see the paper if you want more details
  - L_2/Euclidean norm as in ridge regression (we'll chat more about this in ML lecture)

Synthetic Difference-in-Differences

Choosing Time Weights: ensure pre and post-treatment periods are balanced for control units

Choose **time weights** $(\hat{\lambda}_0, \hat{\lambda})$ that yield arg min of

$$\sum_{i=1}^{N_{co}} \left(\underbrace{\lambda_0 + \sum_{t=1}^{T_{pre}} \hat{\lambda}_t Y_{it}}_{Weighted\ Average\ in\ Pre-Period} - \underbrace{\frac{1}{T_{post}} \sum_{t=T_{pre}+1}^{T} Y_{it}}_{Average\ in\ Post-Period} \right)^2$$

- No regularization in time weights
  - Allows for correlation over time for same unit, but not across units (beyond the systematic component in a latent factor model)

Synthetic Difference-in-Differences

Once unit and time weights (and ζ) are obtained, estimate $\hat{\tau}^{sdid}$ as arg min of

$$\sum_{i=1}^{N}\sum_{t=1}^{T}(Y_{it}-\mu-\alpha_i-\beta_t-W_{it}\tau)^2\hat{\omega}_i^{sdid}\hat{\lambda}_t^{sdid}$$

Note that without weights, this is just the TWFE Diff-in-Diff solution:

$$\sum_{i=1}^{N}\sum_{t=1}^{T}(Y_{it}-\mu-\alpha_i-\beta_t-W_{it}\tau)^2$$

Synthetic Diff-in-Diff Application

Now that we know how it's working, let's try it out by looking at the impact of California's Proposition 99, which introduced a 25 cent tax per pack of cigarettes sold from January 1989 onward.

```
data(california_prop99)
```

Synthetic Diff-in-Diff Application

We *could* evaluate the impact of Prop 99 on cigarette sales per capita using a typical TWFE Diff-in-Diff:

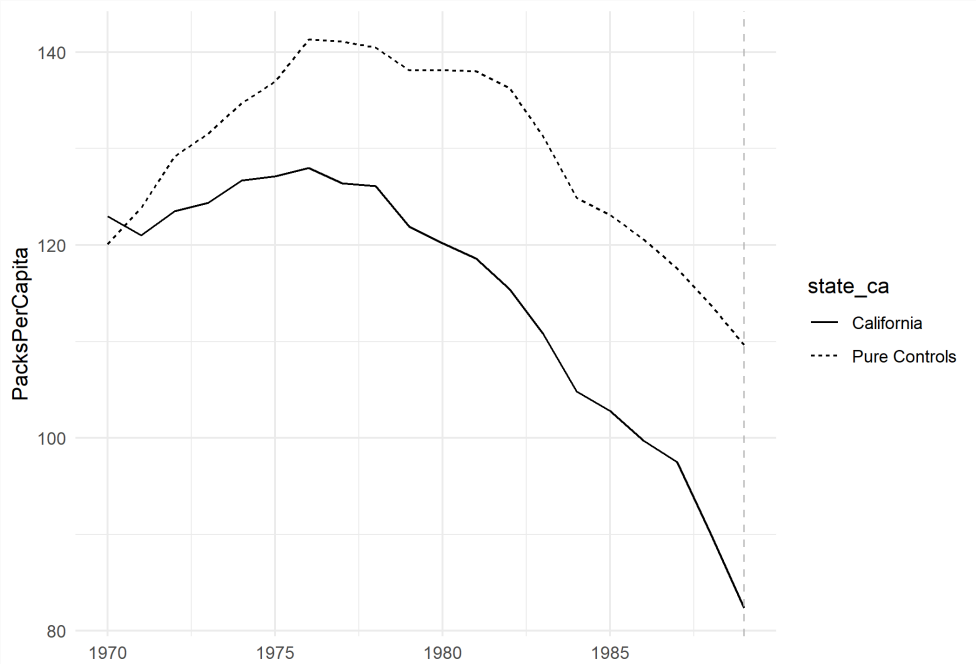
```
did ← feols(PacksPerCapita ~ treated | State + Year, data =
california_prop99, cluster = ~ State)
summary(did)
```

```
## OLS estimation, Dep. Var.: PacksPerCapita
## Observations: 1,209
## Fixed-effects: State: 39,  Year: 31
## Standard-errors: Clustered (State)
##         Estimate Std. Error  t value   Pr(>|t|)
## treated -27.3491    2.80238 -9.75925 6.6913e-12 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
## RMSE: 11.5     Adj. R2: 0.870229
##              Within R2: 0.032671
```

Synthetic Diff-in-Diff Application

Looking at parallel trends:

```
mutate(california_prop99, state_ca = ifelse(State == "California", "California", "Pure Controls")) %>%
group_by(state_ca, Year) %>%
  summarise(PacksPerCapita = mean(PacksPerCapita)) %>%
  filter(Year ⩽ 1989) %>%
  ggplot() +
  geom_line(aes(x = Year, y = PacksPerCapita, linetype = state_ca)) +
  geom_vline(aes(xintercept = 1989), linetype = "dashed", color = "gray70", alpha = 0.7) +
  theme_minimal()
```



Synthetic Diff-in-Diff Application

Let's use the **synthdid** package to estimate a diff-in-diff using a synthetic California as the counterfactual.

First, we'll need to do some se the `panel.matrices()` function to set up the data

- Balanced panel (have)
- Simultaneous adoption (have mechanically)

```
synth_ca_prep ← panel.matrices(
  panel = as.data.frame(california_prop99), # the dataframe
  unit = "State", # unit column (name or column #)
  time = 2, # time column (name or column #)
  outcome = 3, # outcome var (name or column #)
  treatment = "treated", # treatment var (name or column #)
  treated.last = TRUE # sort treated units to be at bottom
)
```

Synthetic Diff-in-Diff Application

Now compute the synthetic Diff-in-Diff estimate with `synthdid_estimate()`:

```
sdid ← synthdid_estimate(Y = synth_ca_prep$Y, # outcome var
                         N0 = synth_ca_prep$N0, # number of control
units (N_co)
                         T0 = synth_ca_prep$T0 # number of pre-
treatment periods (T_pre)
                         )
sdid
```

```
## synthdid: -15.604 +- NA. Effective N0/N0 = 16.4/38~0.4. Effective T0/T0
= 2.8/19~0.1. N1,T1 = 1,12.
```

Which yields a much smaller treatment effect estimate than the Diff-in-Diff (-27.3).

Synthetic Diff-in-Diff Application

Looking at control unit weights ω:

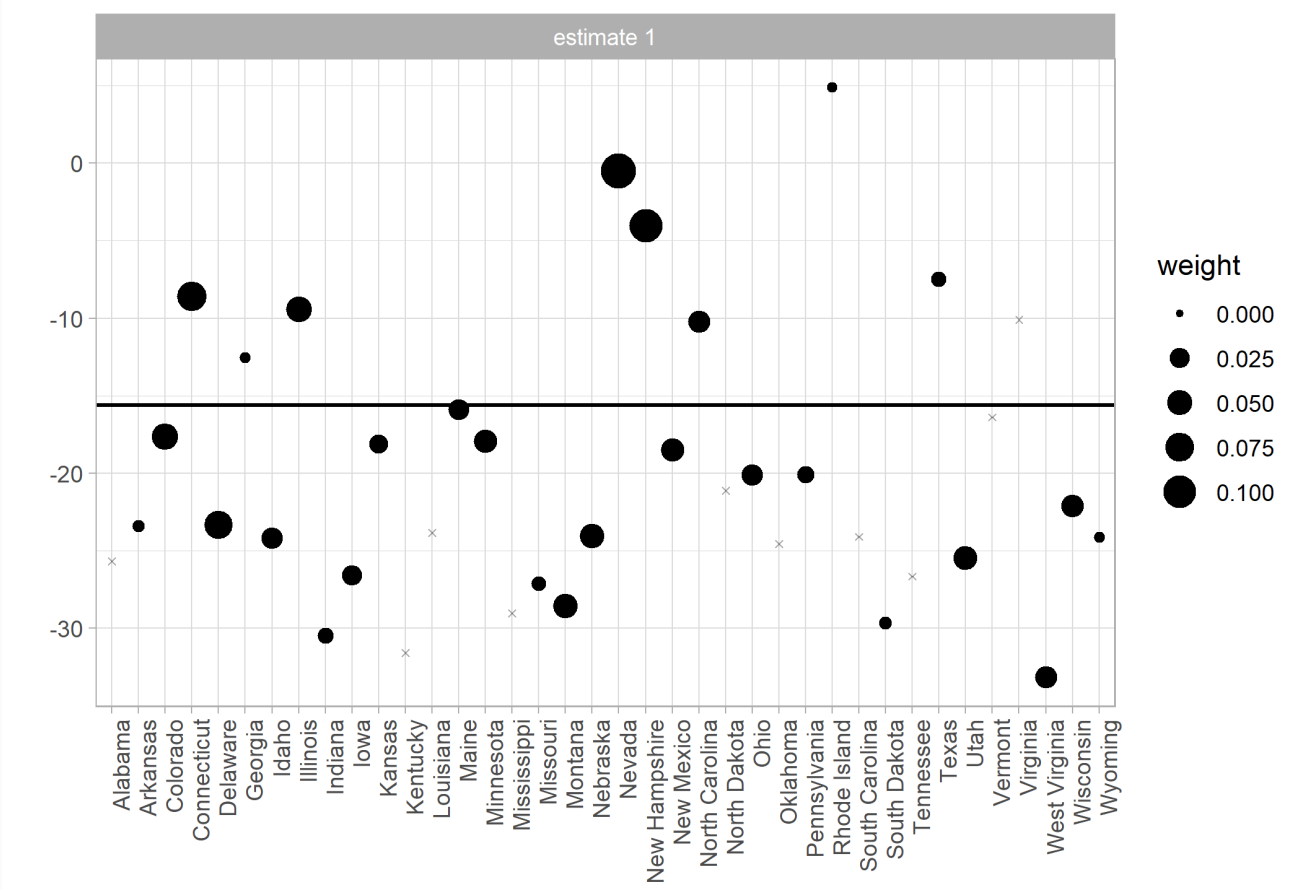
```
synthdid_controls(sdid)
```

```
##                estimate 1
## Nevada         0.12448923
## New Hampshire  0.10504758
## Connecticut    0.07828729
## Delaware       0.07036812
## Colorado       0.05751279
## Illinois       0.05338782
## Nebraska       0.04785319
## Montana        0.04513521
## Utah           0.04151766
## New Mexico     0.04056827
## Minnesota      0.03949464
## Wisconsin      0.03666708
## West Virginia  0.03356911
## North Carolina 0.03280518
## Idaho          0.03146821
```

Synthetic Diff-in-Diff Application

Plotting the control unit weights ω:

```
synthdid_units_plot(sdid)
```



Synthetic Diff-in-Diff Application

And the time weights λ

```
summary(sdid)$periods
```

```
##      estimate 1
## 1988      0.427
## 1986      0.366
## 1987      0.206
```

Synthetic Diff-in-Diff Application

Note that with only one treated unit we can only use the (untrustworthy) `placebo` method to get standard errors, by calling `vcov()` on our `synthdid_estimate` object:

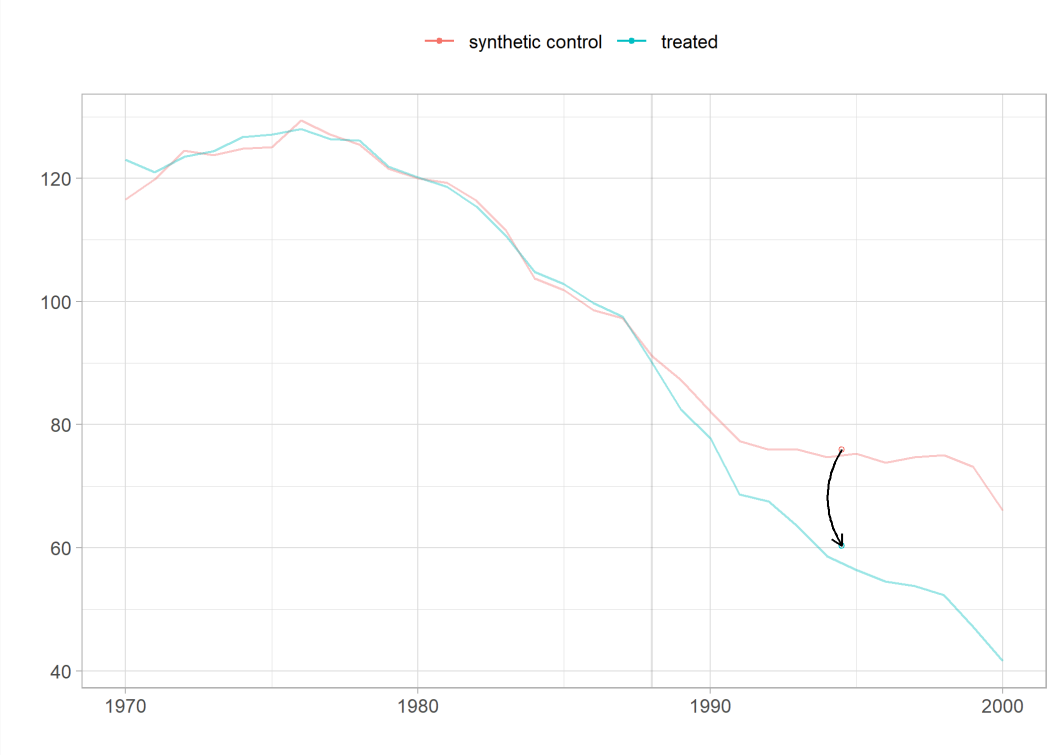
```
se ← sqrt(vcov(sdid, method='placebo'))
sprintf('95%% CI (%1.2f, %1.2f)', sdid - 1.96 * se, sdid + 1.96 * se)
```

```
## [1] "95% CI (-35.55, 4.34)"
```

Synthetic Diff-in-Diff Application

We can look at pre-treatment parallel trends by overlaying the two series:

```
synthdid_plot(sdid, overlay = 1)
```



Synthetic Diff-in-Diff Application

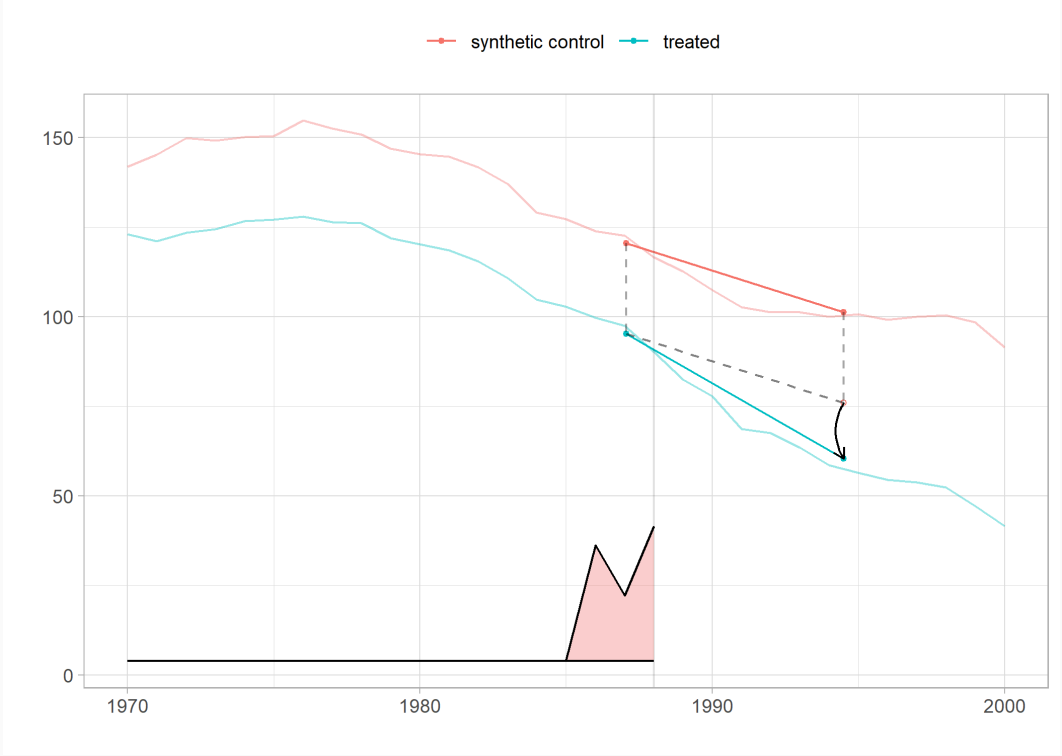
The built-in `synthdid` plot (a `ggplot` object) displays a lot of information by default.

- Point and line segments for simple 2x2 Diff-in-Diff comparison
- Time period weights (bottom red line)
- Customizable further with `theme()` and **other adjustments**

Synthetic Diff-in-Diff Application

The built-in `synthdid` plot (a `ggplot` object) displays a lot of information by default.

```
synthdid_plot(sdid)
```



Synthetic Diff-in-Diff Application

Comparing to canonical SCM and Diff-in-Diff reveals the differences well:

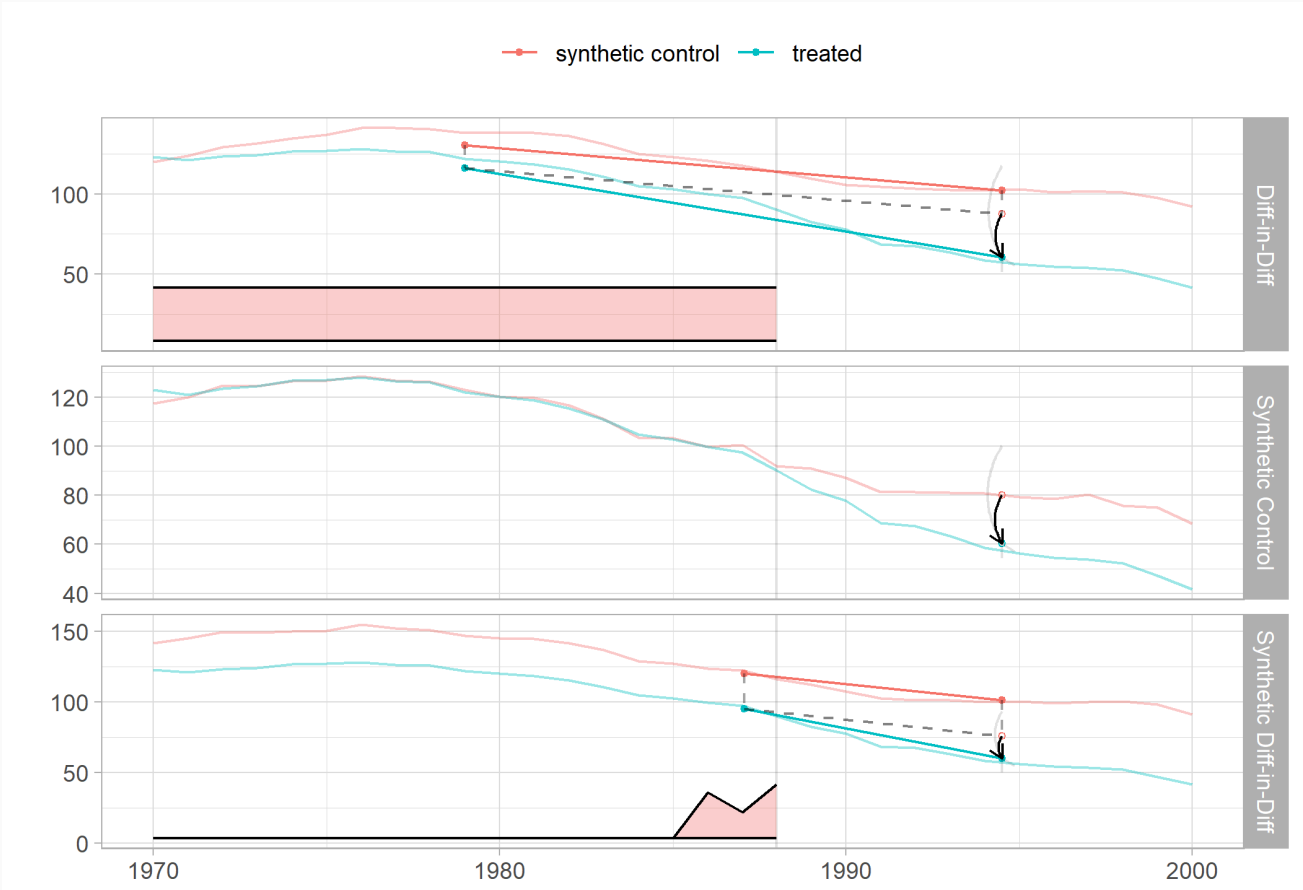
```
est_sc ← sc_estimate(Y = synth_ca_prep$Y, N0 = synth_ca_prep$N0, T0 =
synth_ca_prep$T0)
est_did ← did_estimate(Y = synth_ca_prep$Y, N0 = synth_ca_prep$N0, T0 =
synth_ca_prep$T0)
estimates ← list(est_did, est_sc, sdid)
names(estimates) = c('Diff-in-Diff', 'Synthetic Control', 'Synthetic
Diff-in-Diff')
print(unlist(estimates))
```

```
##           Diff-in-Diff      Synthetic Control Synthetic Diff-in-Diff
##              -27.34911              -19.61966              -15.60383
```

Synthetic Diff-in-Diff Application

Comparing to canonical SCM and Diff-in-Diff reveals the differences well:

```
synthdid_plot(estimates, se.method='placebo')
```



Staggered Adoption

The methods in **synthdid** easily extend to

- Controlling for covariates in the second step regression
- Cases of more than one unit adopting simultaneously.

One current limitation is lack of direct support for **staggered adoption**

Until they add native support, you can use the **Ssynthdid** package

```
# install.remotes("remotes")
remotes::install_github("tjhon/ssynthdid")
```

Partially Pooled Synthetic Control

Partially Pooled Synthetic Control

An alternate but related estimator is the **partially pooled synthetic control method (PPSCM)** of **Ben-Michael, Feller, and Rothstein (2021)**, which extends canonical SCM to the **many-treated unit** and **staggered adoption** cases

They additionally

- Incorporate unit-level intercepts and balancing on covariates
  - Equivalent to balancing on residualized unit-level outcomes
- Estimate dynamic treatment effects
- Obtain standard errors through bootstrapping/jackknifing
  - No formal guarantees for consistency/bias though

Partially Pooled Synthetic Control

More formally:

Suppose you have a panel with

- T time periods
- N units
  - Some units $j = 1..., J$ receive treatment, potentially at different times T_i
  - Non-zero number of pure controls N_0 with $T_i = \infty$
    - Can be block adoption or staggered adoption

You observe

- The time-varying outcome Y_{it},
- A binary treatment indicator W_{it},
- and (optionally) time-varying covariates X_{it}

Partially Pooled Synthetic Control

Assumptions:

- Stable treatment and no interference across units (SUTVA)
- Prior to treatment, a unit's potential outcomes are equal to its never-treated potential outcomes
- **No Anticipation:**

$$Y_{it}(s) = Y_{it}(\infty) \text{ for } t < s, \text{ with treatment time } s$$

Partially Pooled Synthetic Control

Assumptions:

- All treated units are observed for at least several pre-periods and several post-periods
  - Needed to ensure sufficient identification in unbalanced event time
- Can express the data generating process as following
  1. Following a time-varying $AR(L)$ process:

     $$Y_{it}(\infty) = \sum_{\ell=1}^{L} \rho_{t\ell} Y_{it-\ell}(\infty) + \epsilon_{it}, \text{ or}$$

     - Rules out correlation between treatment timing and noise terms for any period
  2. Composed of time-varying latent factors and time-invariant unit loadings: $Y_{it}(\infty) = \phi_i \cdot \mu_t + \epsilon_{it}$
     - Rules out correlation between treatment timing and noise terms *after* treatment
  3. Noise term ϵ_{it} are sub-Gaussian random variables

Partially Pooled Synthetic Control

If we want to extend canonical SCM to the **many-treated** case, we could take one of two approaches.

1. Separate SCM: estimate a separate SCM for each treated unit

- i.e. minimize pre-treatment imbalance for each treated unit separately
- Can lead to poor mean fit, biasing the ATE

2. Pooled SCM: estimate one SCM for average of treated units

- i.e. minimize the average pre-treatment imbalance across all treated units
- Can achieve strong average fit but obtain poor unit-specific treatment effects

The Solution: "pool" the two estimators, improving on each approach in isolation

Partially Pooled Synthetic Control

Choose SCM weights to minimize the weighted average of the pooled and unit-specific pre-treatment balance:

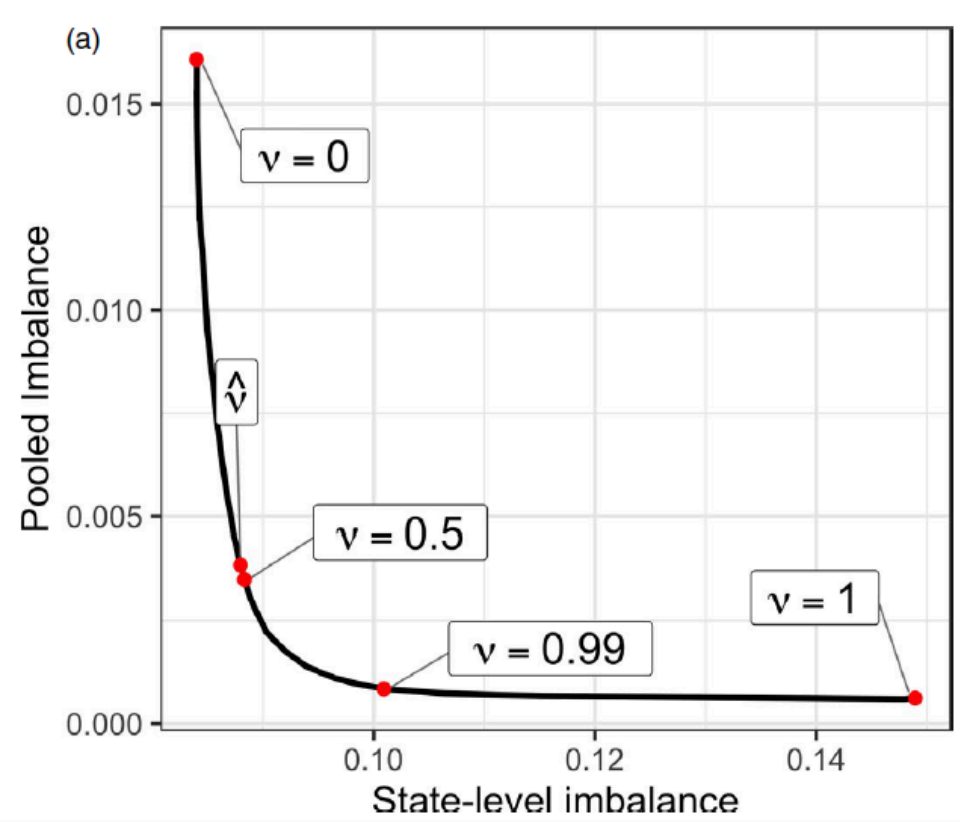
$$\min_{\Gamma \in \Delta^{scm}} \nu \underbrace{(\tilde{q}^{pool}(\Gamma))^2}_{\substack{\text{Normalized} \\ \text{pooled} \\ \text{imbalance}}} + (1-\nu) \underbrace{(\tilde{q}^{sep}(\Gamma))^2}_{\substack{\text{Normalized} \\ \text{separate} \\ \text{imbalance}}} + \underbrace{\lambda ||\Gamma||_F^2}_{\substack{\text{penalize sum of} \\ \text{squared weights}}}$$

- $\hat{q}^{sep}$, $\hat{q}^{pool}$ the (normalized) root mean square of separate and pooled pre-treatment fit
- $\lambda ||\Gamma||_F^2$ a penalty term (as in SCM)
- ν the hyperparameter determining the degree of "partial pooling"
  - $\nu = 0 \Rightarrow \text{Separate SCM}$
  - $\nu = 1 \Rightarrow \text{Pooled SCM}$
  - $0 < \nu < 1 \Rightarrow \text{Partially-Pooled SCM}$

Partially Pooled Synthetic Control

So, what ν should we use?

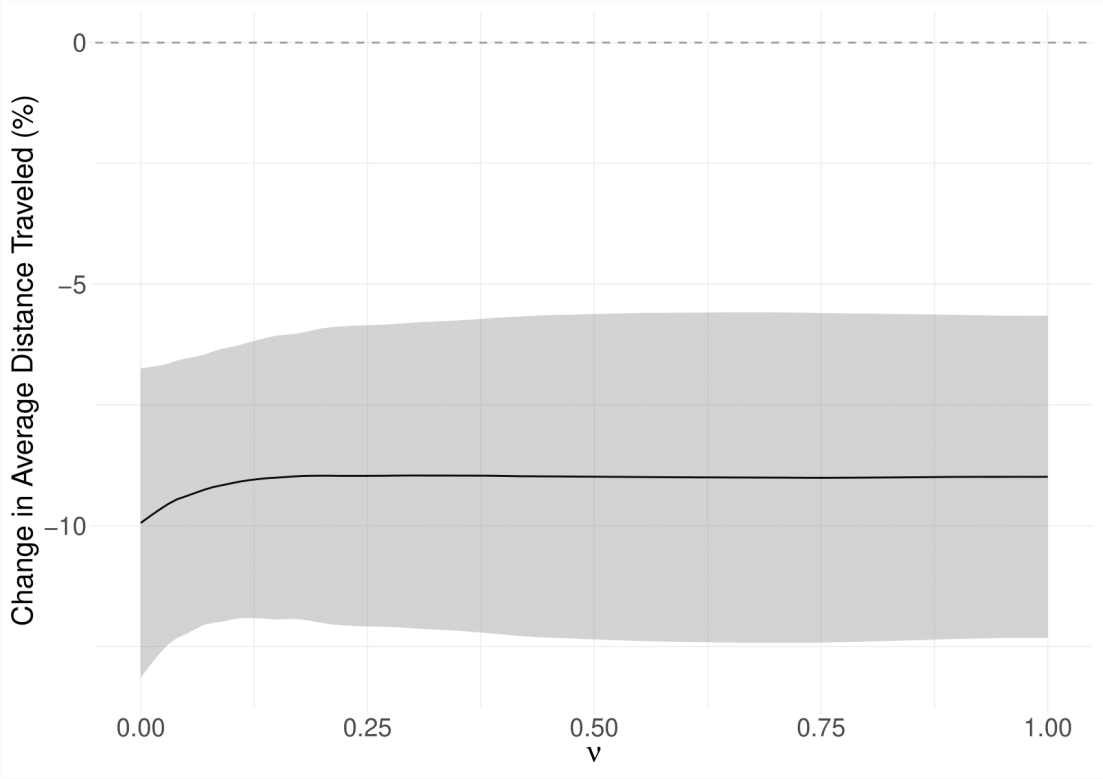
Turns out the relationship between pooled and separate imbalance is **highly convex**, with even slight interior ν offering large improvements.



Partially Pooled Synthetic Control

So, what ν should we use?

In my experience, overall ATEs are highly similar across the space of ν



Quantifying Uncertainty

Partially Pooled SCM relies upon a **wild bootstrap** for inference, but no guarantees are provided for ensuring its (asymptotic) validity.

Fortunately, **Cattaneo et al. (2025) REStat**† formalize methods for both **in-sample** and **out-of-sample** uncertainty in the multiple treatment unit staggered adoption SCM case that guarantee coverage probabilities non-asymptotically.

- Also: provide for estimation + inference of unit-time, unit-average, and time-average effects
- Also also: allow for a wide range of alternate weighting constraints, multiple features, and covariate adjustment
- Also also also: built an r package **scpi** to implement all this

† Downside: it's quite new so there **aren't a lot of examples** yet.

Synthetic Control Prediction Intervals

Cattaneo et al. SCPI

Cattaneo et al. start from a general SCM framework:

- Consider J_0 never-treated units and J_1 eventually-treated
- Let $Y_{j,T_j+k}^{T_j}(1)$ be the observed outcome at event time k for a unit j that receives treatment at time T_j
- We want to impute the unobserved counterfactual $\widehat{Y}_{j,T_j+k}^{T_j}(0)$ as a weighted combination of units in the donor pool N_0
  - Constant α_j allows matching on *trends* rather than imposing identical *values* in each period

$$\widehat{Y}_{j,T_j+k}^{T_j}(0) = \alpha_j + \sum_{i \in N_0} \hat{\omega}_{ij} Y_{i,T_j+k}$$

Uncertainty Quantification

While Cattaneo et al. offer a lot of flexibility in terms of how to obtain $\hat{\omega}$, the big achievement is in quantifying **multiple forms of uncertainty**.

To do this, they view the true potential outcomes (and the treatment effects) as **random objects**, which allows them to develop **finite-sample prediction intervals** for imputed counterfactual outcomes using **probability concentration methods**.

- $\Rightarrow$ refer to τ_{jk} as a **predictand** and its uncertainty quantified with **prediction intervals**

Uncertainty Quantification

Consider a set of "true" synthetic control weights $\mathbf{w^j}$ for treated unit j.

Its prediction error (difference between the unit's predicted and true counterfactual) in post-adoption period ℓ is a function of both in- and out-of-sample uncertainty:

$$\underbrace{\hat{Y}_{j,\ell}(0) - Y_{j,\ell}(0)}_{\tau_{j,\ell}} = \underbrace{\mathbf{Y}_{N_0,\ell}^{\mathsf{T}} \left(\hat{\mathbf{w}}^{\mathbf{j}} - \mathbf{w}^{\mathbf{j}} \right)}_{\text{in-sample uncertainty}} - \underbrace{u_{j,\ell}}_{\text{out-of-sample uncertainty}}$$

Uncertainty Quantification

$$\underbrace{\hat{Y}_{j,\ell}(0) - Y_{j,\ell}(0)}_{\tau_{j,\ell}} = \underbrace{\mathbf{Y}_{N_0,\ell}^{\mathsf{T}} \left(\hat{\mathbf{w}}^{\mathbf{j}} - \mathbf{w}^{\mathbf{j}} \right)}_{\text{in-sample uncertainty}} - \underbrace{u_{j,\ell}}_{\text{out-of-sample uncertainty}}$$

In-Sample Uncertainty

- Arises from the construction of the synthetic control weights
  - Dependent on the quality of the available conditioning information
- Estimate through principled simulation-based method of Cattaneo, Fend, and Titiunik (2021)

Out-of-Sample Uncertainty

- Arises from misspecification and additional noise in the period and unit-specific stochastic error
- Estimated through probabilistic simulation with bounds estimated with
  1. Non-asymptotic bounds,
  2. A location-scale model, or
  3. Quantile regression

SCPI Application

So how does the Cattaneo et al. approach actually work? Let's use the **scpi** package to revisit mobility responses to stay-at-home mandates.

Recall the setup:

- States adopted stay-at-home mandates on different days (staggered adoption)
- Observe many pre-period dates (Feb 24 to Mar 19)
- Observe many post-adoption dates (# varies by state, from 22 to 42)

SCPI Estimation: 2 Steps

1. Prepare the data for estimation

- `scdata()` for single treated unit
- `scdataMulti()` for multiple treated units
- Define treatment/outcome, specify treated/donor units
- Features to balance on, covariates for adjustment
- Desired effect of interest (unit-time, time, unit)
- More settings! (anticipation, cointegration, feature-specific constants)

2. Estimate the model

- `scest()` for synthetic control point estimation
  - Choice of constraint
  - Separate or pooled estimation
- `scpi()` for point estimation + prediction intervals/uncertainty estimation

† Partially pooling is possible too by specifying the appropriate `V.mat` argument

SCPI Application

We've got multiple treated units + staggered adoption, so we'll set up the data with `scdataMulti()`:

- Default: use outcome as only feature, no covariate adjustment, no feature-specific constant, no weight constraint, unit-time estimand
- Instead, let's
  - Balance on distance traveled (`cadt`) and the daily caseloads (`cases`)
  - Add constant + linear trend in travel distance, linear trend in caseloads
  - Constrain weights with an L1-L2 norm

Prepare the Data

First, we use `scdataMulti()` to prepare the data:

- Specify everything here *except* the feasibility set on the constraint

```
sah_scpi ← drop_na(sah, cadt, cases, post_treat)
scdm_adt ← scdataMulti(
  df = sah_scpi, # the data frame containing the data
  id.var  = "state", # character name of variable with unit IDs (can be
numeric or character)
  time.var = "date", # character name of time variable (can be numeric,
character, or Date from as.Date())
  outcome.var = "cadt", # character name of outcome variable
  treatment.var = "post_treat", # character name of treatment variable
  features = list(c("cadt", "cases")),
  cov.adj = list("cadt" = c("constant", "trend"), "cases" = c("trend")),
# constand + linear trend in travel distance, linear trend in caseloads
  effect = "unit-time", # type of effect desired (default is unit-time)
  verbose = TRUE # prints additional info to console
)
```

Estimate with Prediction Intervals

Next, let's obtain the point estimates and bound the in- and out-of-sample error with `scpi()`...

but first we have to **specify the contraint set** for our SCM weights!

Constraint Set

In **scpi**, we have a *ton* of control over the constraints we impose on $\hat{\boldsymbol{\omega}}$ through the `w.constr` argument. We can specify it in a couple ways.

1. Provide a List of Custom Constraint Arguments

- `p`: specify the constrained norm of w (`"no norm", "L1", "L2", "L1-L2"`)
- `dir`: the direction on the constraint $||w||_p$ (`"==", "<=", "==/<="`)
- `lb`: the lower bound on the weights (`0` or `-Inf`)
- `Q, Q2`: size of the constraints (any real #)

i.e. for canonical SCM:

```
list(p = "L1", dir = "==", Q = 1, lb = 0)
```

Feasibility Set

In **scpi**, we have a *ton* of control over the constraints we impose on $\hat{\boldsymbol{\omega}}$ through the `w.constr` argument. We can specify it in a couple ways.

2. Refer to Pre-Specified Constraint Profiles by Name

Canonical SCM (nonnegative, sum to one):

$$\mathcal{W} = w \in \mathbb{R}_+^J : ||w||_1 = 1$$

- Shorthand: `list(name = "simplex", Q = 1)`

Feasibility Set

In **scpi**, we have a *ton* of control over the constraints we impose on $\hat{\boldsymbol{\omega}}$ through the `w.constr` argument. We can specify it in a couple ways.

2. Use Pre-Specified Constraint Profiles

OLS (Hsiao et al. 2012):

$$\mathcal{W} = \mathbb{R}^J$$

- Shorthand: `list(name = "ols")`
- Equivalent to `list(p = "no norm", lb = -Inf, dir = NULL, Q = NULL)`

Feasibility Set

In **scpi**, we have a *ton* of control over the constraints we impose on $\hat{\boldsymbol{\omega}}$ through the `w.constr` argument. We can specify it in a couple ways.

2. Use Pre-Specified Constraint Profiles

LASSO (Chernozhukov et al. 2021):

$$\mathcal{W} = w \in \mathbb{R}^J : ||w||_1 \leq 1$$

- Shorthand: `list(name = "lasso", Q = 1)`
- Equivalent to `list(p = "L1", lb = -Inf, dir = "⩽", Q = 1)`

Feasibility Set

In **scpi**, we have a *ton* of control over the constraints we impose on $\hat{\boldsymbol{\omega}}$ through the `w.constr` argument. We can specify it in a couple ways.

2. Use Pre-Specified Constraint Profiles

Ridge (Amjad et al. 2018):

$$\mathcal{W} = w \in \mathbb{R}^J : ||w||_2 \leq Q$$

- Shorthand: `list(name = "ridge")`
- Equivalent to `list(p = "L2", lb = -Inf, dir = "⩽", Q = 1)`

Feasibility Set

In **scpi**, we have a *ton* of control over the constraints we impose on $\hat{\boldsymbol{\omega}}$ through the `w.constr` argument. We can specify it in a couple ways.

2. Use Pre-Specified Constraint Profiles

L1-L2 (Arkhangelsky et al. 2021, Cattaneo et al. 2025):

$$\mathcal{W} = w \in \mathbb{R}_{+}^{J} : ||w||_1 = 1, ||w||_2 \leq Q$$

- Shorthand `list(name = "L1-L2", Q = 1)`
- Equivalent to `list(p = "L1-L2", lb = 0, Q = 1, Q2 = 1, dir = "=/<=")`

SCPI Estimation

Next, let's *actually* obtain the point estimates and bound the in- and out-of-sample error with `scpi()` [†]:

```
tic()
fit_scpi ← scpi(data = scdm_adt, # prepared scd/scDM data object
                          w.constr = list(name = "L1-L2", Q = 1, Q2 =
1), # constraint set on the weights
                          V = "separate", # form of weighting matrix
(separate or pooled)
                          sims = 2000,  # simulations per treated unit
in error quantification
                          e.method = "all",
                          cores = future::availableCores() - 4)
toc()
saveRDS(fit_scpi, "output/fit_scpi.rds")
```

† This takes a while. If all we cared about were the point estimates, we could just use `scest()` and skip the computationally-intensive prediction interval estimation.

SCPI

Our `scpi` object has a *lot* of stuff, including three main results list items

- `data`: the input data used for estimation
- `est.results:` the point estimates
  - Estimated weights, SC unit(s)' time paths, residuals
  - Obtained from a nested run of `scest()`!
- `inference.results`: all the inference-related results
  - In- and out-of-sample uncertainty prediction intervals, estimated bounds

SCPI

Get a glance at everything with `print()`:

```
fit_scpi ← read_rds("output/fit_scpi.rds")
print(fit_scpi)
```

```
##
## Synthetic Control Prediction - Results
##
## Active donors:
## AL AZ CA CO CT DE FL GA ID IL IN KS KY LA MA MD ME MI MN MO MS MT NC NH
NJ NM
##  1  1  1  1  1  6  1  1  4  1  1  4  4  1  1  1  4  1  2  1  1  5  1  6
1  6
## NV NY OH OR PA RI SC TN TX VA VT WA WI WV
##  1  1  1  3  1  2  1  1  1  1  6  1  1  3
##
## Coefficients:
##    AL AZ CA CO CT    DE FL GA    ID IL IN    KS    KY LA MA MD    ME MI
MN
## AR  0  0  0  0  0 0.021  0  0 0.050  0  0 0.000 0.015  0  0  0 0.017 0
```

Looking at the summary:

```
summary(fit_scpi)
```

```
## Print and summary methods are not available for scpi when data are
processed with scdataMulti()!
```

... or not

Synthetic Control

Manually looking at our synthetic control weights for Delaware:

```
##   State Weight
## 1    OK 0.5712
## 2    NE 0.2303
## 3    IA 0.1096
## 4    ND 0.0550
## 5    AR 0.0206
## 6    WY 0.0134
## 7    SD 0.0000
## 8    UT 0.0000
```

scplot()/scplotMulti()

While it's possible to access the inference objects directly, they're not in a readily-plottable format.

Easier: use the `scplot()` or `scplotMulti()` functions to generate the plot data/plots

- `scplot()` for single treated units
- `scplotMulti()` for multiple treated units

scplotMulti()

When `effect = "unit-time"`, `scplotMulti()` facets for each treated unit, which can get unintelligible fast (as in our case):

```
scplotMulti(fit_scpi)
```

```
## $plot_out_gau
```



scplotMulti()

... but if we follow the function's warning we can save out the formatted plot data + build our preferred plot manually:

- Alternative: access manually from `inference.results$CI` objects, do lots of data wrangling

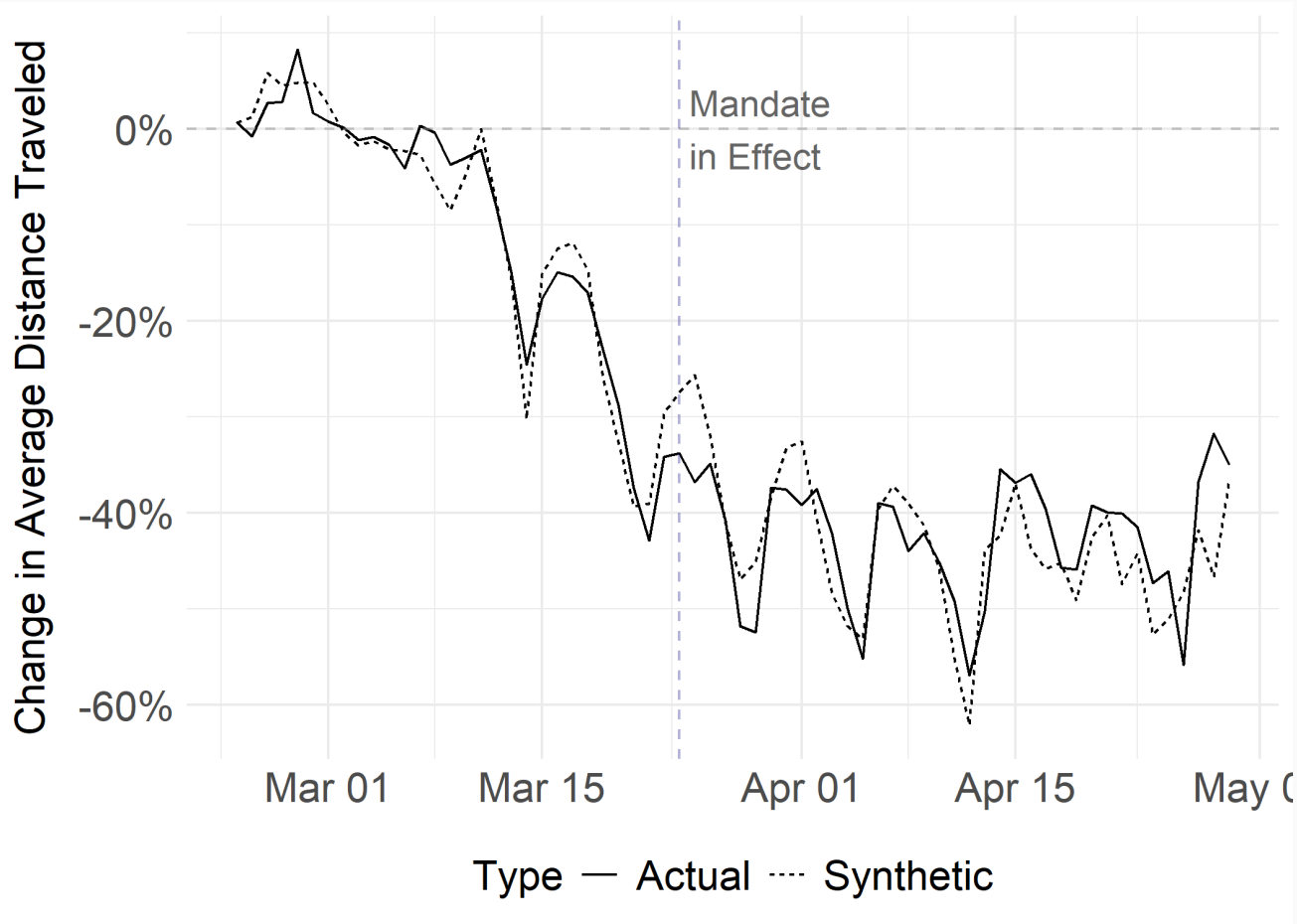
```
# Obtaining the series for treated units and their synthetic controls
scplotMulti(fit_scpi, type = "series",
            save.data = "output/scpi_plot_series")
```

```
# load it in
load("output/scpi_plot_series.RData")

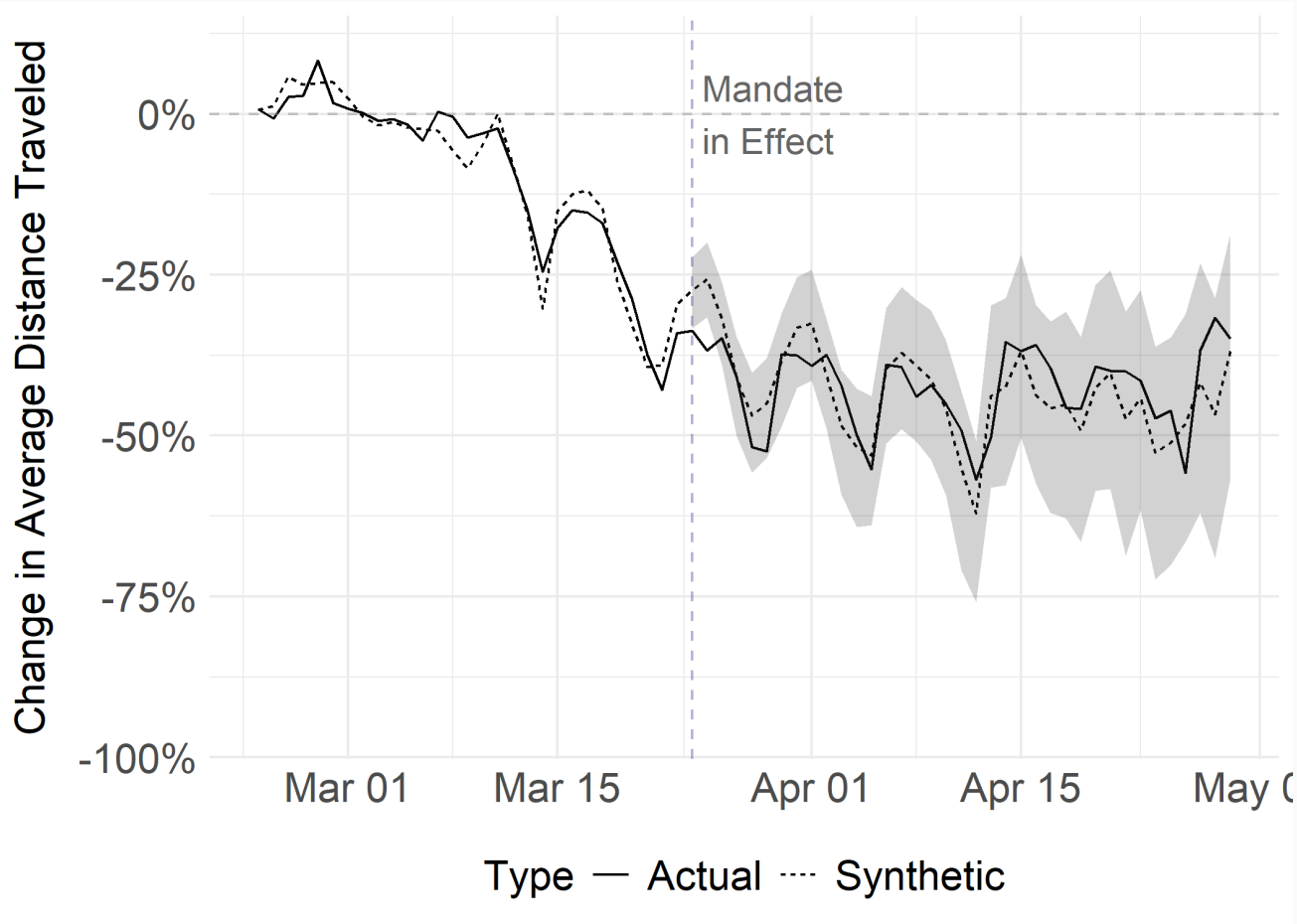
# Filtering to just DE
de_df_series ← filter(toplot, ID == "DE")
```

Pre-Treatment Fit

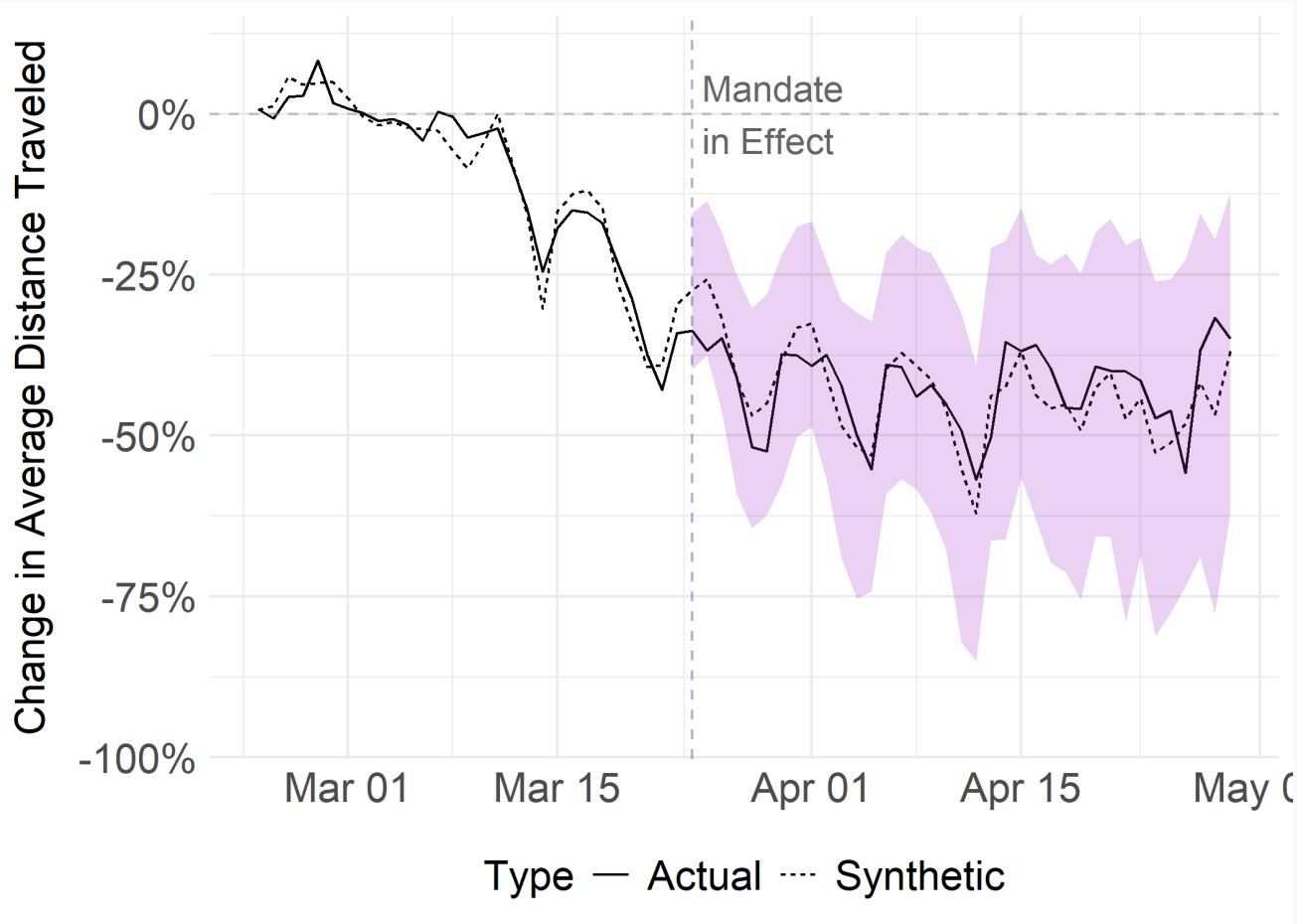
Looking at pre-treatment balance and post-treatment path for Delaware:



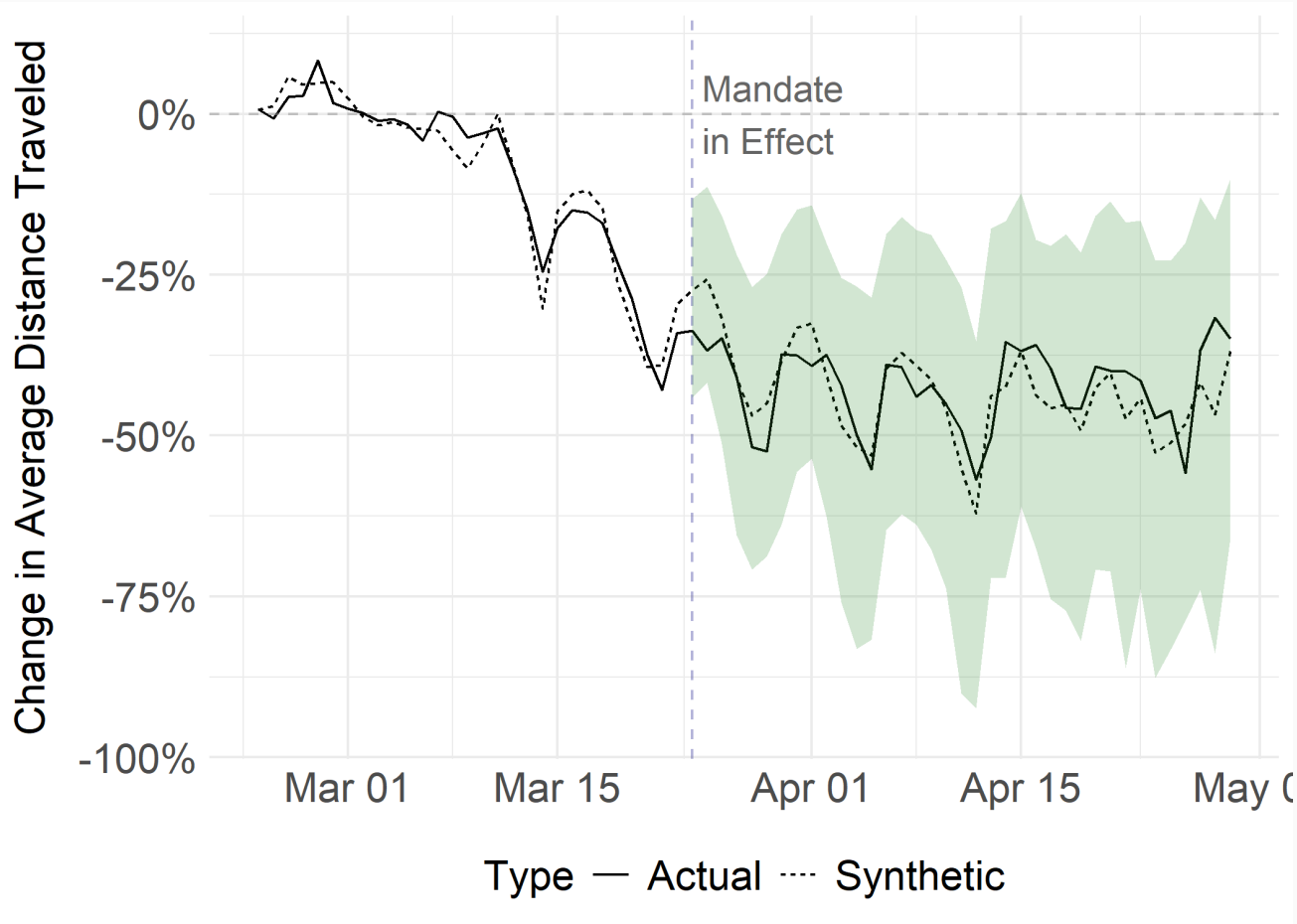
Prediction Intervals (In-Sample)



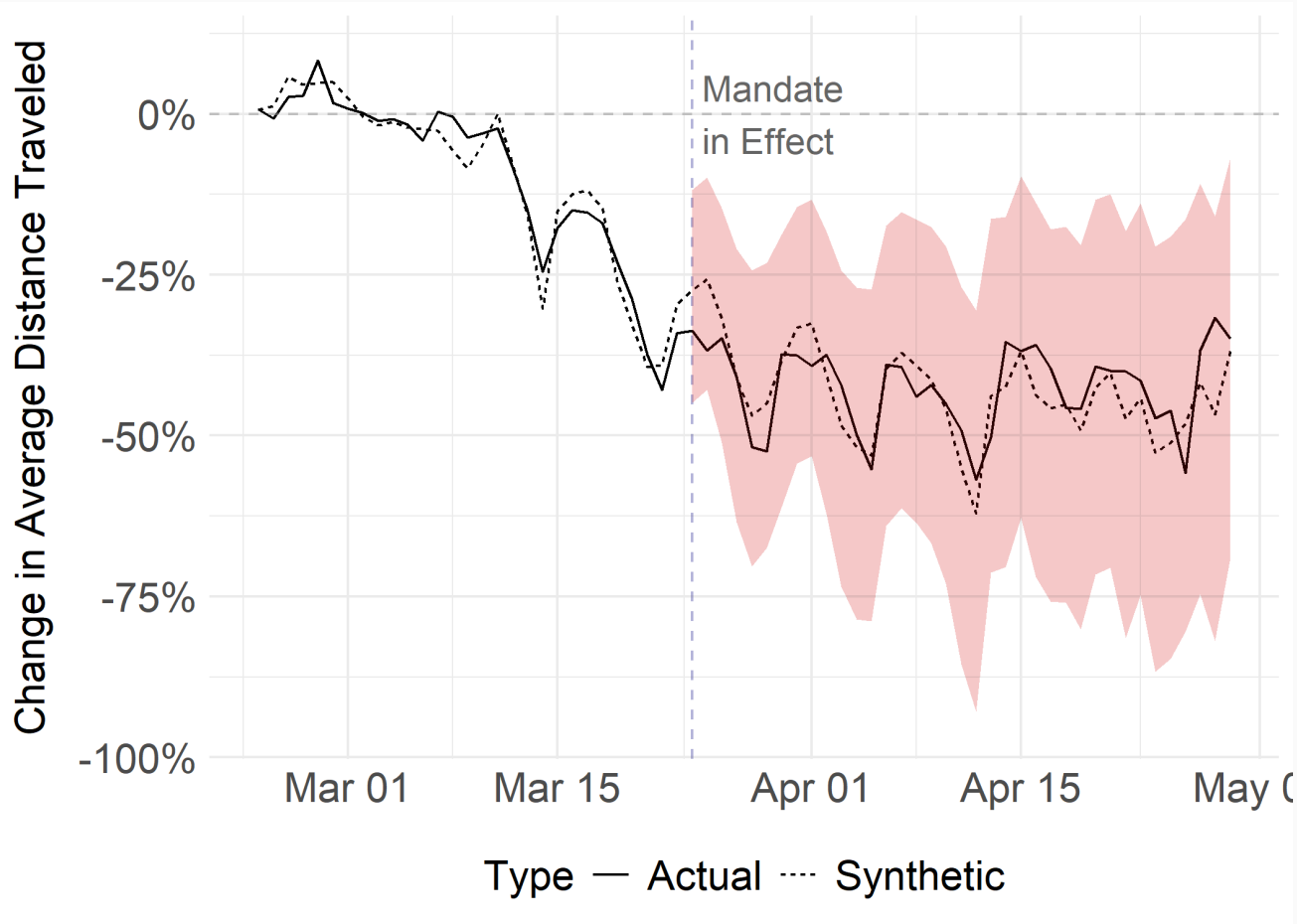
Prediction Intervals (Gaussian)



Prediction Intervals (Location Scale)



Prediction Intervals (Quantile Reg.)



Treatment Effects

Looking at the treatment effects for Delaware:

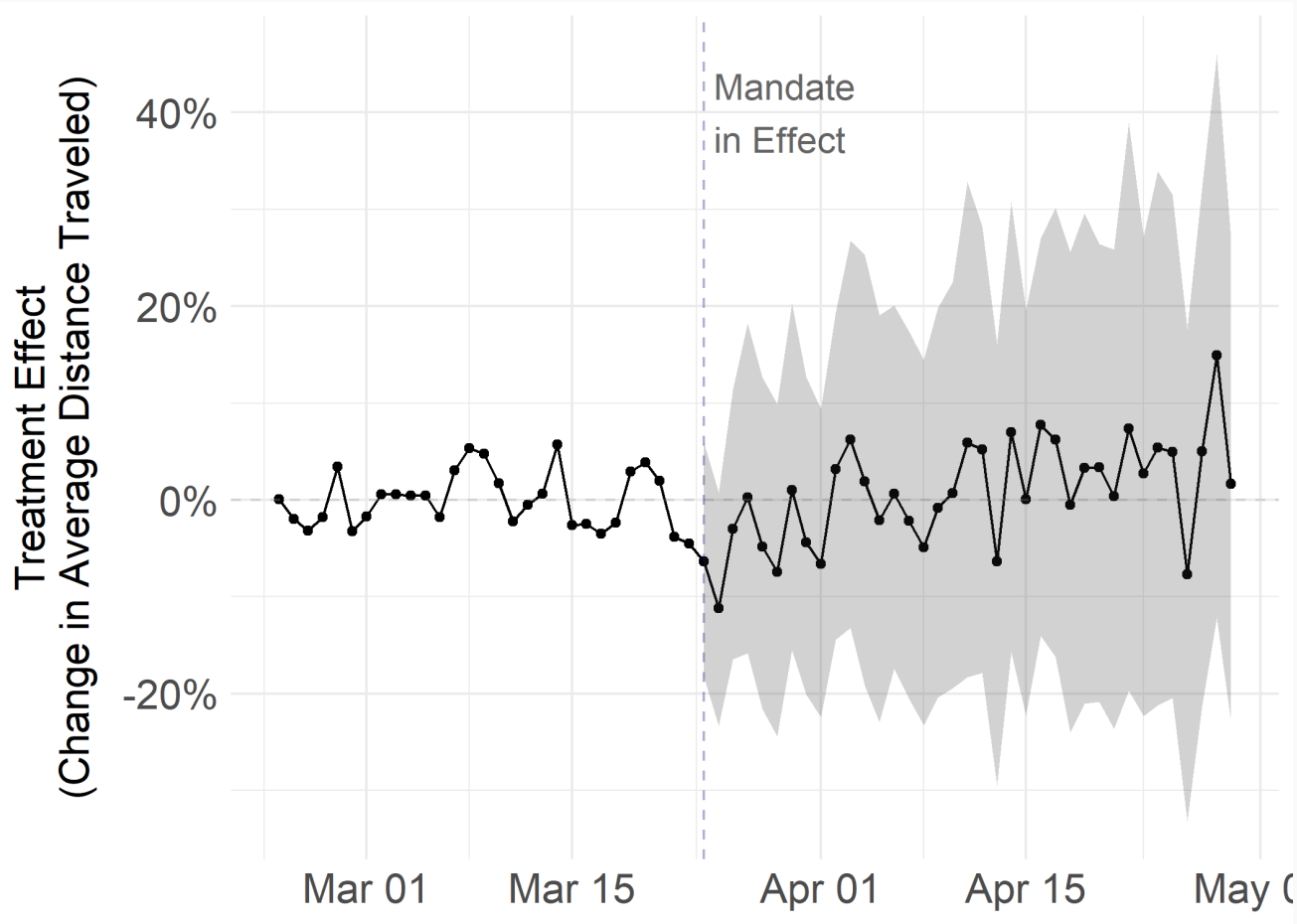
- obtained from `scplotMulti()` with `type = treatment`:
- stored in `inference.results$bounds`

```
# Obtaining the series for treated units and their synthetic controls
scplotMulti(fit_scpi, type = "treatment",
            save.data = "output/scpi_plot_tau")
```

```
load("output/scpi_plot_tau.RData")

# Filtering to just DE
de_df_tau ← filter(toplot, ID == "DE")
```

Treatment Effects



Time or Unit-Averaged Effects

While we often care about event study style time-specific, unit-specific (TSUS) treatment effects $\hat{\tau}_{j,T_j+k} = Y^{T_j}_{j,T_j+k}(1) - \widehat{Y}^{T_j}_{j,T_j+k}(0)$, we might also care about

Time-Specific, Unit-Averaged (TSUA) Effects

$$\hat{\tau}_{\cdot k} = \frac{1}{|N_1|} \sum_{j \in N_1} \hat{\tau}_{jk}$$

- For a given period k, aggregate across all $j \in N_1$ treated units

Time-Averaged, Unit-Specific (TAUS) Effects

$$\hat{\tau}_{j\cdot} = \frac{1}{T - T_j + 1} \sum_{k=0}^{T-T_j} \hat{\tau}_{jk}$$

- For a given treated unit j, average across all its post-treatment periods $T_j, \ldots, T_j + 1, \ldots T$

Time or Unit-Averaged Effects

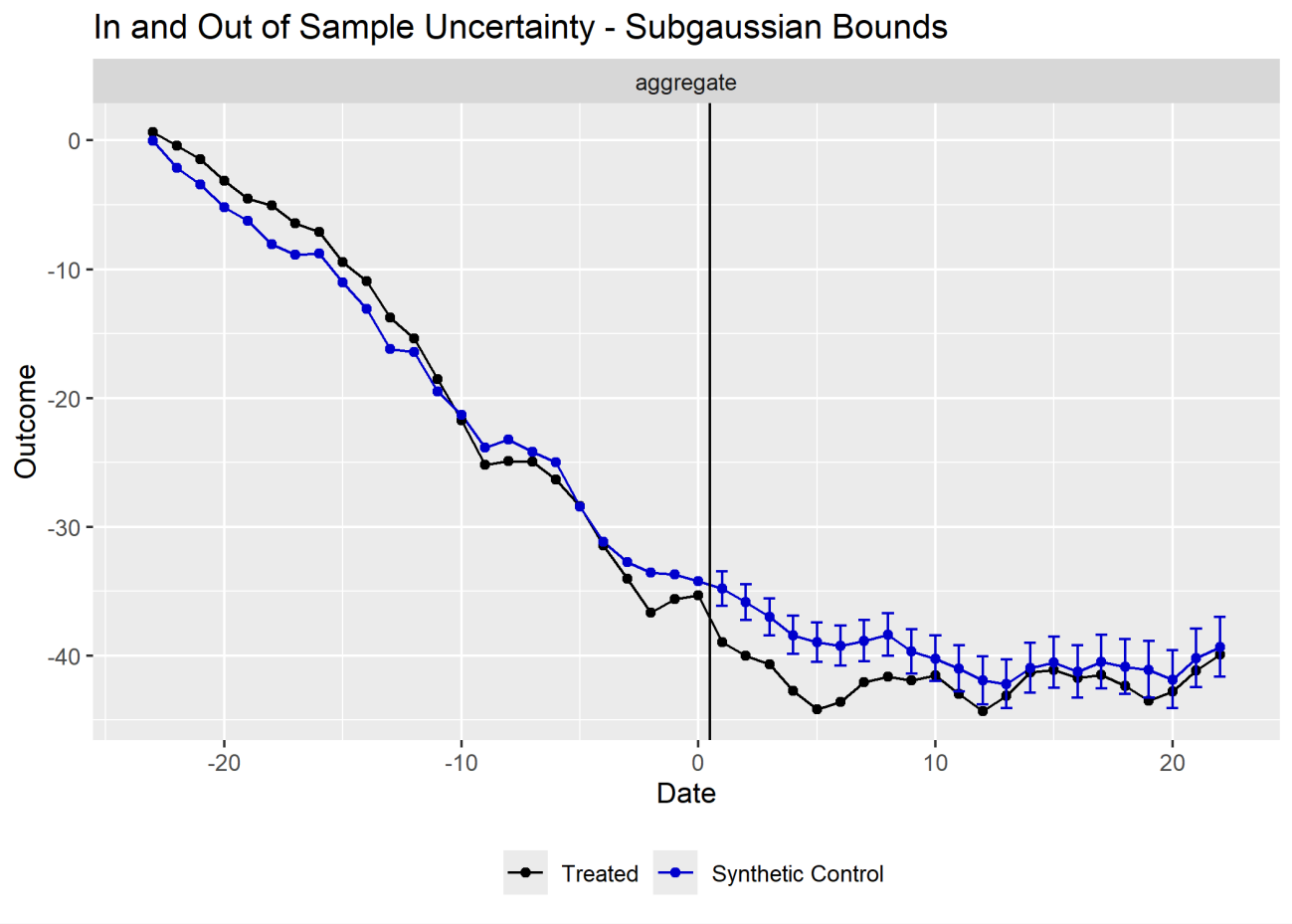
We can easily obtain point estimates + prediction intervals for either predictand using the `effect` argument in `scdataMulti()`!

For example, the time-specific, unit-averaged (TSUA) effects:

```
# Time-Specific, Unit-Averaged (TSUA) Effects
scdm_adt_tsua ← scdataMulti(
  df = sah_scpi, # the data frame containing the data
  id.var  = "state", # character name of variable with unit IDs (can be
numeric or character)
  time.var = "date", # character name of time variable (can be numeric,
character, or Date from as.Date())
  outcome.var = "cadt", # character name of outcome variable
  treatment.var = "post_treat", # character name of treatment variable
  features = list(c("cadt", "cases")),
  cov.adj = list("cadt" = c("constant", "trend"), "cases" = c("trend")),
# constand + linear trend in travel distance, linear trend in caseloads
  effect = "time" #
)
fit_scpi_tsua ← scpi(data = scdm_adt_tsua, # prepared scd/scDM data
```

scplotMulti()

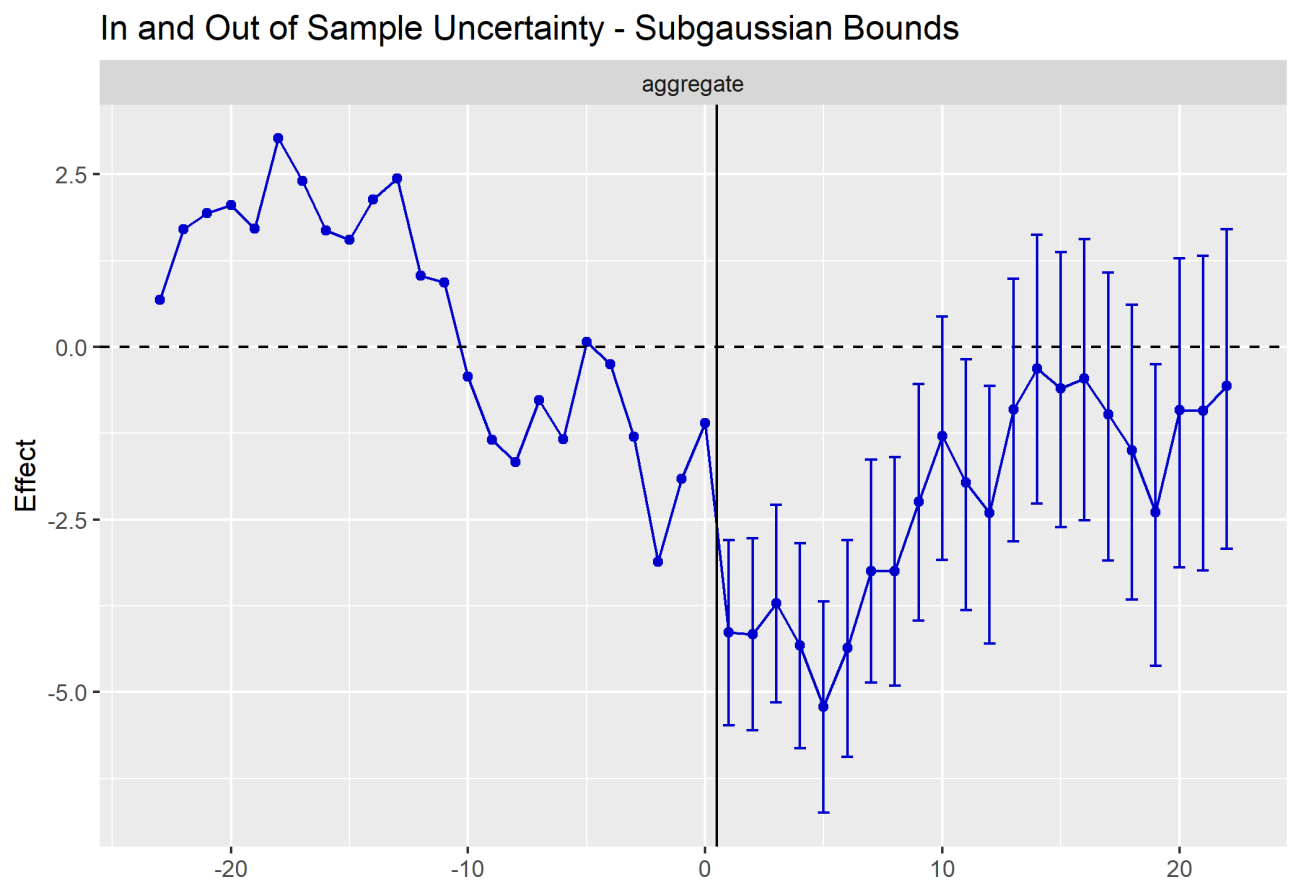
When there's only one estimated synthetic control (average of all treated units), the built-in `scplotMulti()` function yields a clean plot:



scplotMulti()

Or the treatment effect:

```
scplotMulti(fit_scpi_tsua, type = "treatment")$plot_out_gau
```



SCM Assumptions

SCM Assumptions

We've seen that SCM approaches can yield improved parallel trends and better balance. Why is this?

A: Rather than require us to directly specify the functional form of proxies for the unobservables, they automatically test all linear combinations of the provided "input features"

Q: That's great, but what are the *right* input features?

A: **Arkhangelsky and Hirshberg (2023)** nicely point out that we should take advantage of all the information we have that **contains information on relevant unobservables**.

- Past outcomes are the most likely candidate for this
- If we have time-varying controls that *also* contain helpful information, then we should include those too!

SCM Assumptions

Q: So then if we have variables that we believe contain sufficient information to fully proxy for the unobservables, we're good, right?

A: Unfortunately **no**; there are still some cases where SC methods will be **biased**.

1. **Dynamic selection**
2. **Heterogeneously persistent time-varying shocks**

SCM Struggles

1. Dynamic Selection: Adoption of treatment based on past outcomes

Consider time t and a group of units that haven't adopted the treatment

- **SC Assumptions:** these units are a **good control group** for the period t **contemporaneous** effect

For **dynamic effects:** need good controls in future periods $t+1, \ldots$

- **Problem:** not being treated currently $\Rightarrow$ outcomes in $t+1, \ldots$ should necessarily be systematically different

$\Rightarrow$ without additional adjustments, estimates for $t+1, \ldots$ will be **biased**

SCM Struggles

2. Heterogeneously persistent time-varying shocks

Consider a scenario where potential outcome $Y_{it}(0)$ is a function of

- Unit-level heterogeneity η_i
- Period-specific shock λ_t,
- Some unobserved shock ϵ_{it}.

If the unobserved shock for all units arises from the **same DGP** for all units,

$$\epsilon_{it}^{d} = \epsilon_{it} \; \forall \; i$$

then SC performs with **minimal bias** and **equal/less noise** than TWFE.

However, if there is **heterogeneity** in the persistence of the time-varying shocks, SC methods become **biased**.

SCM Struggles

2. Heterogeneously persistent time-varying shocks

If there is **heterogeneity** in the persistence of the time-varying shocks, SC methods become **biased**.

Suppose some units' potential outcomes depend on ϵ_{it}^1 and others on ϵ_{it}^2

- e.g. AR(1) vs. random walk

Then **SC fails** because there is an **added dimension of permanent heterogeneity**

- ϵ_{it}^1, ϵ_{it}^2 matter for *both* the selection model *and* the conditional mean of counterfactual outcomes
- Linear combo of past outcomes can't distinguish units that come from ϵ_{it}^1 from those from ϵ_{it}^2
- $\Rightarrow$ **bias similar** to TWFE, ***more*** **noise**

SCM Struggles

Documentation of these struggles of SCM are **quite recent** and still limited to working papers.

The bottom line is that, while we're still figuring out the settings where SCM doesn't work, there is a large class of models in which SC methods will likely **outperform traditional TWFE estimators**.

Also, it shows an opportunity for **future research!**

- What if dynamic selection process can be modeled or well-proxied with covariates?
  - i.e. match on propensity model

Partially Pooled Synthetic Control: Application

PPSCM Application

So how does PPSCM actually work? Let's use the **augsynth** package and use PPSCM to revisit mobility responses to stay-at-home mandates.

Recall the setup:

- States adopted stay-at-home mandates on different days (staggered adoption)
- Observe many pre-period dates (Feb 24 to Mar 19)
- Observe many post-adoption dates (# varies by state, from 22 to 42)

PPSCM Application

One advantage to PPSCM is less data prep - we can jump directly to estimation with `augsynth()`

```
augsynth(y ~ treat | weighting covars | approx match covars | exact
         match covars, unit, time, data, n_leads, n_lags)
```

- `formula` requires just Y_{it} and W_{it}
  - Optional weighting covariates, approximate/exact matching covariates
  - `unit/time` the names of unit and time variables
  - can be text, numeric, or dates
  - Don't need manual event time
  - `data` the dataframe
  - `n_leads/n_lags` the number of lead/lag event times to estimate
  - `n_lead` default: # post-treatment dates for last-treated unit (same as our binned event time before)
  - `n_lags` default: balance all periods

PPSCM Application

Estimating with default settings and storing the summary object

- Most everything is hanging out in the `summary()` object

```
if (!require("augsynth"))
remotes::install_github("ebenmichael/augsynth")
pacman::p_load(augsynth)
sah_ppscm ← multisynth(cadt ~ post_treat, state, date,  data = sah,
fixedeff = TRUE)
sum_ppscm ← summary(sah_ppscm)
```

PPSCM Application

Looking at the output shows

- Average ATT and standard error (across all time periods + treated units)
- Imbalances and improvement over pooled/separate SCM

```
sum_ppscm
```

```
##
## Call:
## multisynth(form = cadt ~ post_treat, unit = state, time = date,
##     data = sah, fixedeff = TRUE)
##
## Average ATT Estimate (Std. Error): -6.585  (1.453)
##
## Global L2 Imbalance: 1.136
## Scaled Global L2 Imbalance: 0.347
## Percent improvement from uniform global weights: 65.3
##
## Individual L2 Imbalance: 5.543
## Scaled Individual L2 Imbalance: 0.701
## Percent improvement from uniform individual weights: 29.9
##
```

PPSCM Application

The coefficient table contains two main types of data

1. "Average" period-specific ATT across all treated units (top rows, "Average" Level)
2. Unit-specific ATTs for each period

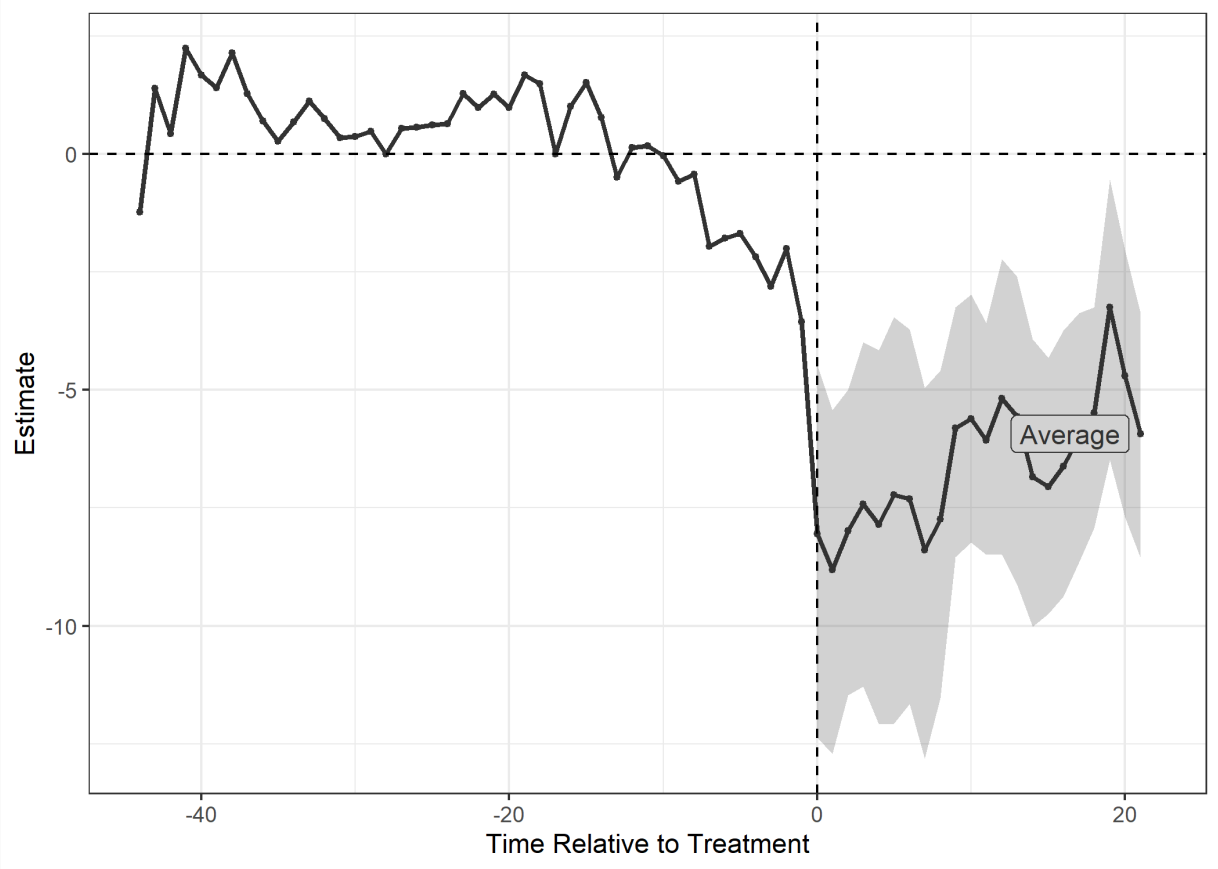
```
att_ppscm ← sum_ppscm$att
att_ppscm
```

```
##      Time   Level      Estimate   Std.Error  lower_bound upper_bound
## 1     -44 Average  -1.231576312  7.91802455 -13.06162820 11.77831123
## 2     -43 Average   1.389608170  5.91902359  -8.60717418 10.05516220
## 3     -42 Average   0.429717019  7.42298367 -12.88118910 13.30605466
## 4     -41 Average   2.236317785  5.18547308  -7.55223980  9.67595146
## 5     -40 Average   1.676579762  5.56036205  -8.97773250  9.03688359
## 6     -39 Average   1.405277301  4.35966319  -7.14517046  6.92136696
## 7     -38 Average   2.138175640  3.15990808  -4.54817326  6.83536116
## 8     -37 Average   1.286093228  2.72357311  -4.32150462  5.84025562
## 9     -36 Average   0.707602719  2.15994947  -3.52937150  4.6694603488 / 102
```

PPSCM Application

Plotting the Average ATT event study:

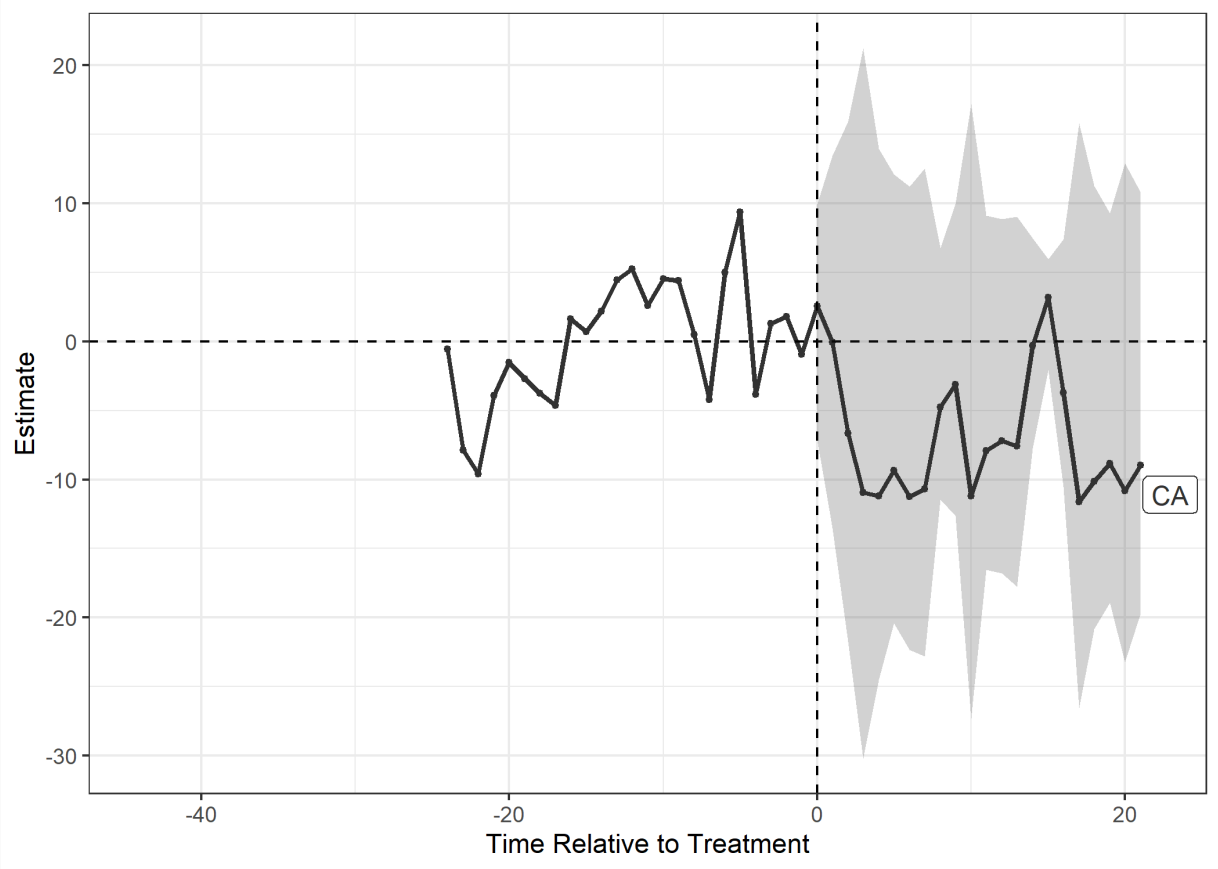
```
plot(sum_ppscm, levels = "Average")
```



PPSCM Application

Or the event study for just CA relative to its synthetic control:

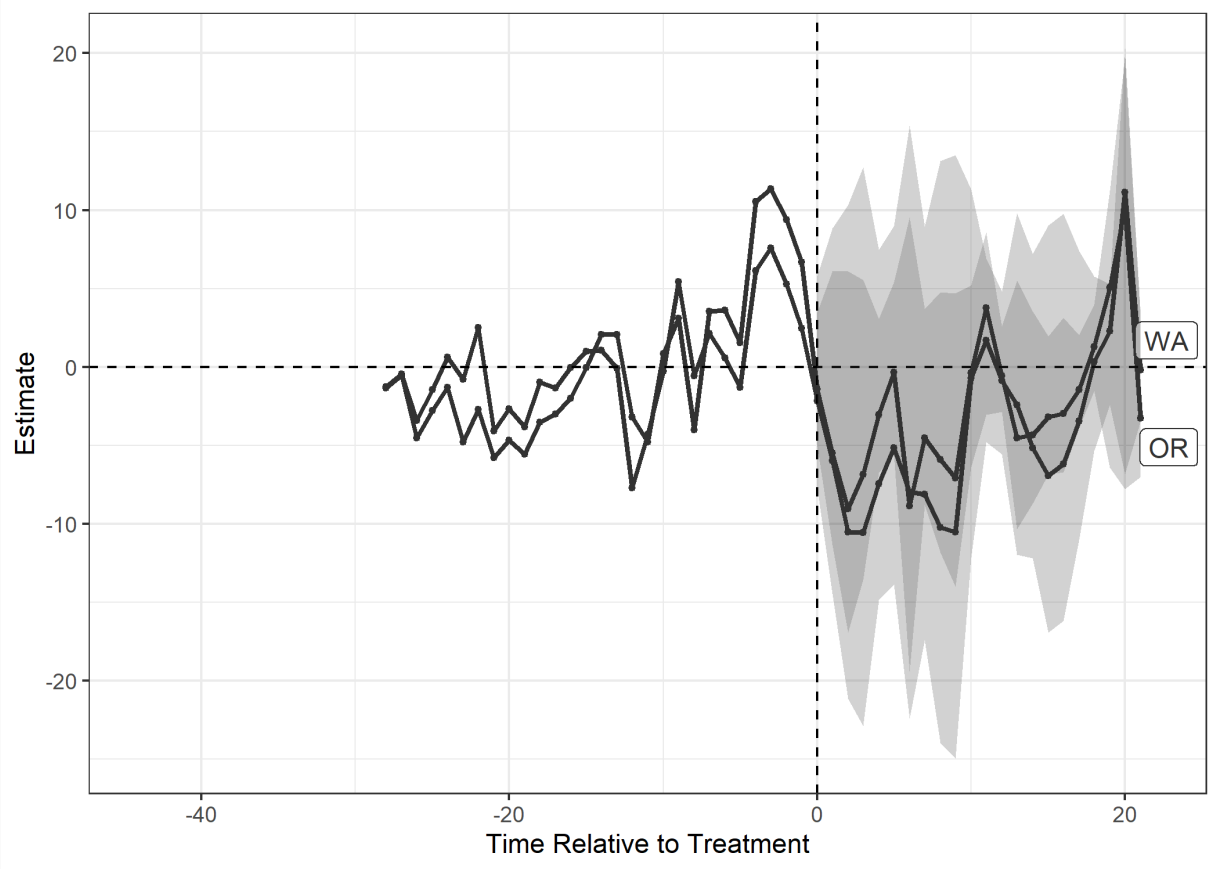
```
plot(sum_ppscm, levels = "CA")
```



PPSCM Application

Or a subset (say, OR and WA):

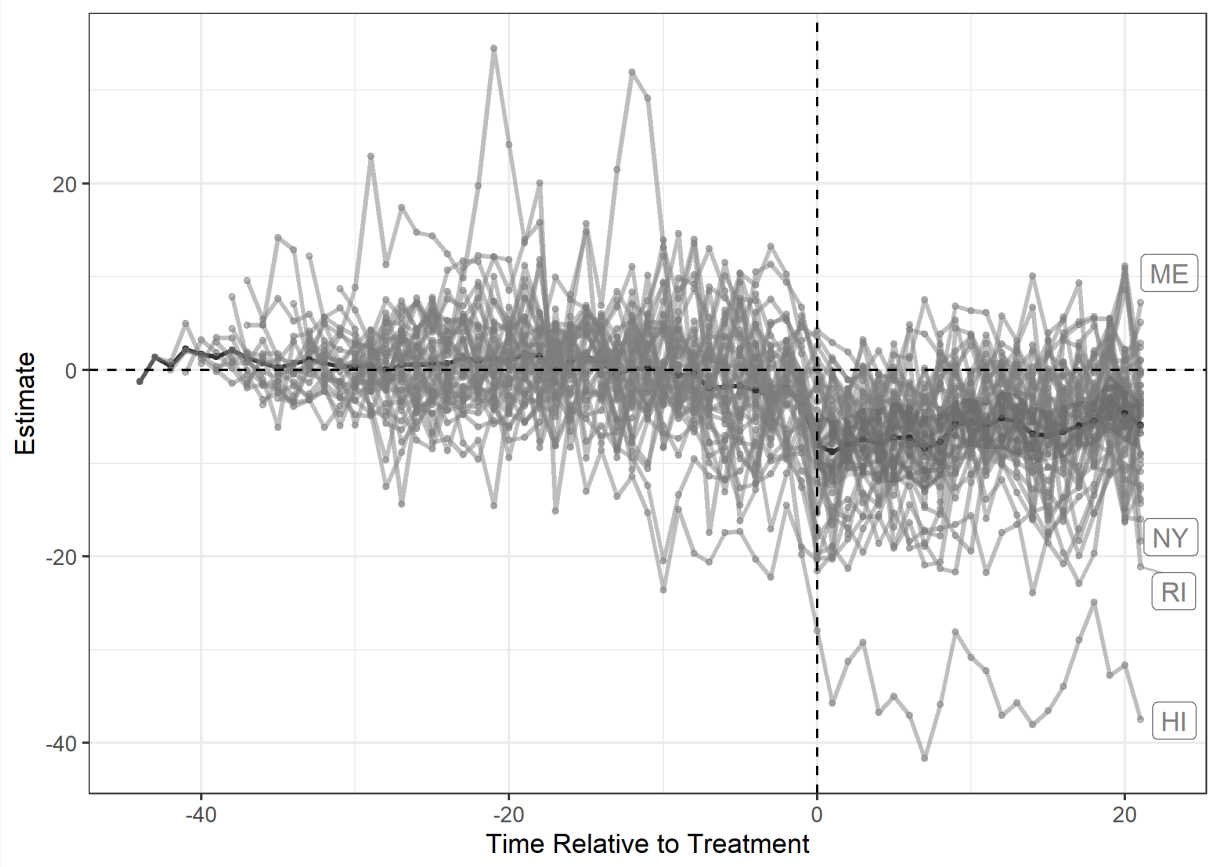
```
plot(sum_ppscm, levels = c("OR", "WA"))
```



PPSCM Application

Or every ATT all at once:

```
plot(sum_ppscm)
```



PPSCM Application

Looking at the weights:

- $N \times N_{tr}$ matrix (51 states + DC by 43 SAH adopters)

```
sah_ppscm$weights
```

```
##        [,1]     [,2]       [,3]         [,4]      [,5]         [,6]
[,7]
## AK 0.000000 0.000000 0.00000000 0.000000e+00 0.0000000 0.000000e+00
0.00000000
## AL 0.000000 0.000000 0.00000000 0.000000e+00 0.0000000 0.000000e+00
0.00000000
## AR 0.000000 0.000000 0.00000000 5.359426e-02 0.0000000 0.000000e+00
0.00000000
## AZ 0.000000 0.000000 0.00000000 0.000000e+00 0.0000000 0.000000e+00
0.00000000
## CA 0.000000 0.000000 0.00000000 0.000000e+00 0.0000000 0.000000e+00
0.00000000
## CO 0.000000 0.000000 0.00000000 0.000000e+00 0.0000000 0.000000e+00
0.00000000
```

PPSCM Application

What is CA (first adopter)'s synthetic control?

```
ca_wts ← sah_ppscm$weights %>%
  as.data.frame() %>%
  select(V1) %>%
  rename(weight = V1)

ca_wts ← mutate(ca_wts, state = rownames(ca_wts)) %>%
  #rename(weight = V1) %>%
  filter(weight ≠ 0)
ca_wts
```

```
##      weight state
## UT 1.000001    UT
```

```
filter(sah, state %in% c("CA", "AR", "IA", "ND", "NE", "OK", "SD", "UT",
"WY")) %>%
  left_join(ca_wts, by = "state") %>%
  mutate(Group = ifelse(state == "CA", "CA", "Synthetic Control (UT)"),
         weight = ifelse(state == "CA", 1, weight)) %>%
```

PPSCM Application

Looking at ν, we see we're about halfway between separate and pooled SCM:

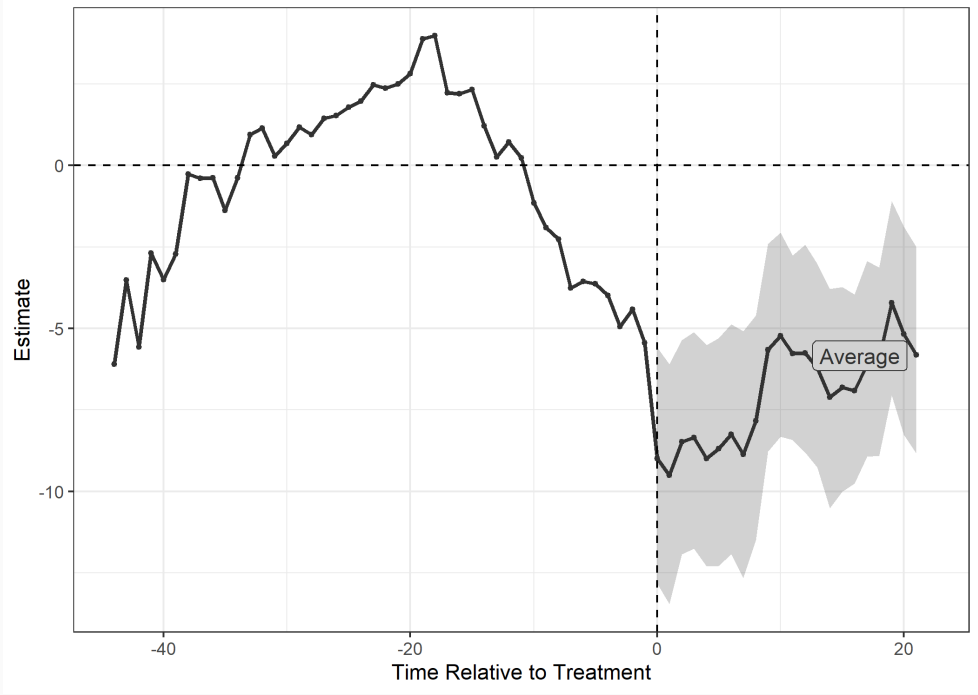
```
sah_ppscm$nu
```

```
## [1] 0.4902226
```

PPSCM Application

By default, the regularization penalty is zero. What happens if we turn it on?

```
sah_ppscm2 ← multisynth(cadt ~ post_treat, state, date,  data = sah, fixedeff = TRUE,
                        lambda = 0.5)
sum_ppscm2 ← summary(sah_ppscm2)
plot(sum_ppscm2, levels = "Average")
```



PPSCM Application

We can also add covariates for

- **Weighting:** covariates to weight on
- **Approx. Matching:** covariates to approximately match on *before* weighting
- **Exact Matching:** covariates to exactly match on *before* weighting

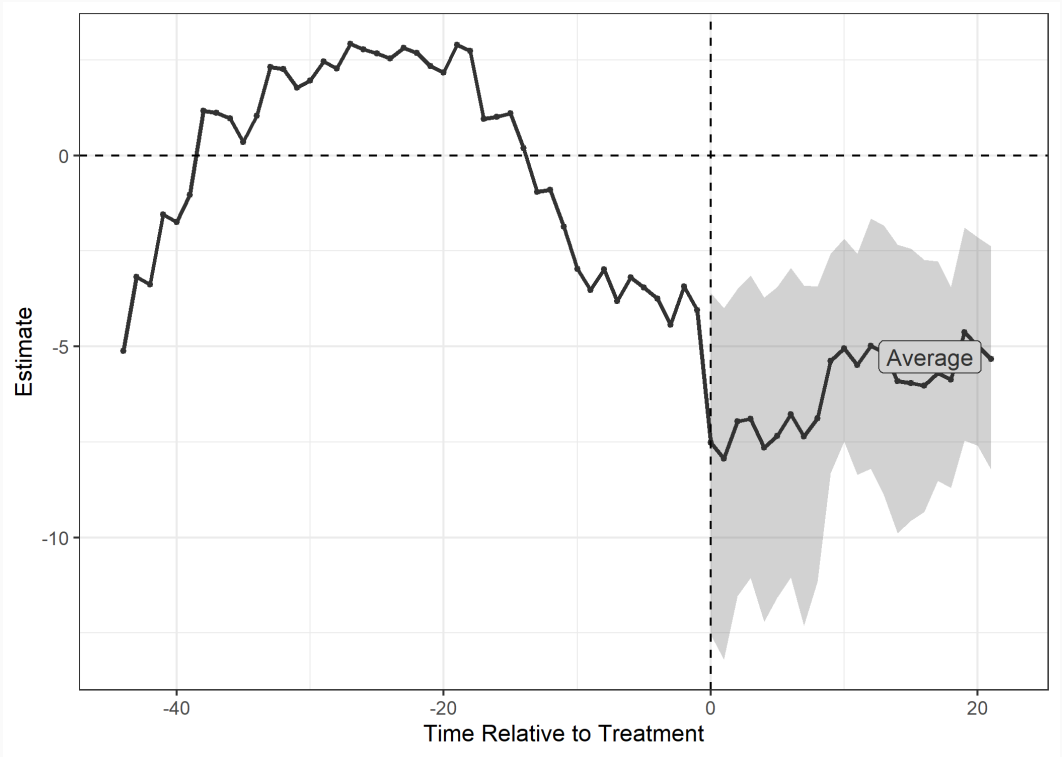
Let's weight on average pre-period values of a few covariates:

```
sah_ppscm3 ← multisynth(cadt ~ post_treat | pct_wfh + pct_pub_trans +
pct_retail + pct_ent_fd + poppm2019 + share_dem, state, date,
                        data = sah, fixedeff = TRUE,
                        lambda = 0.5)
sum_ppscm3 ← summary(sah_ppscm3)
```

PPSCM Application

Looking at balance under this specification:

```
plot(sum_ppscm3, levels = "Average")
```



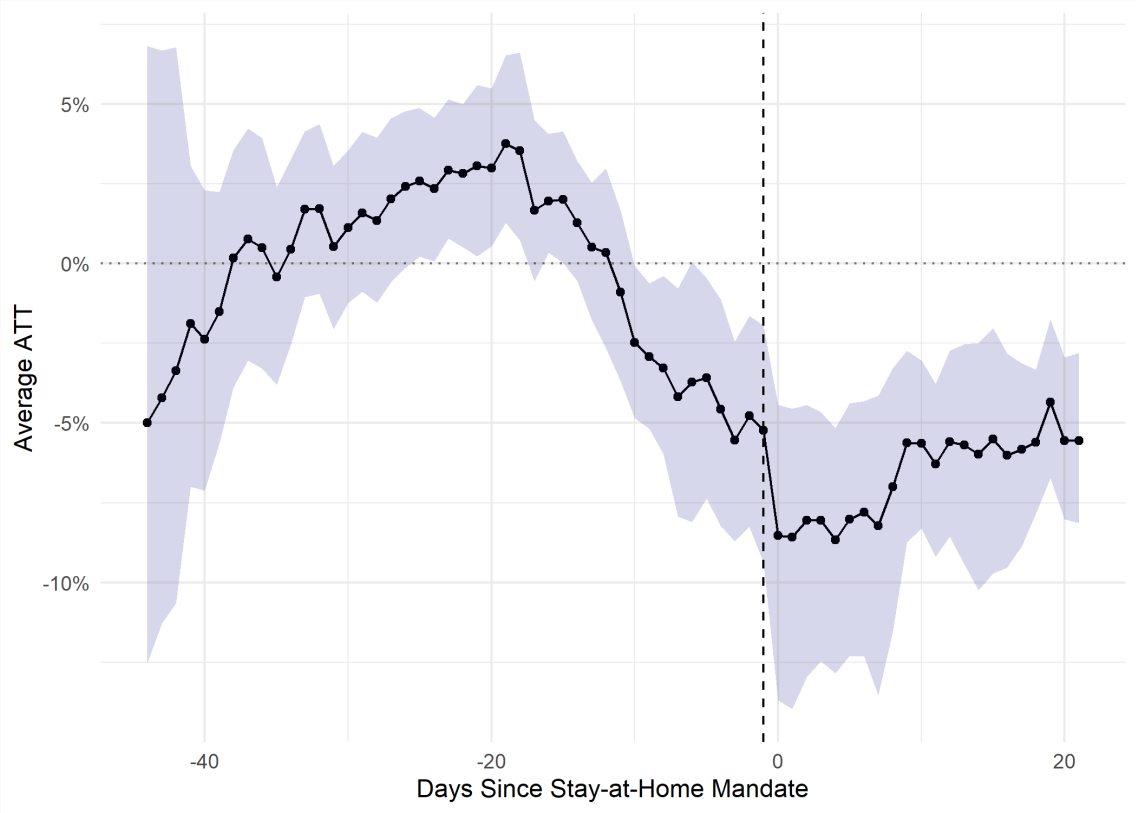
PPSCM Application

Alternatively, we can approximately match on population and democratic vote share in 2016 election using 3 nearest neighbors:

```
sah_ppscm4 ← multisynth(cadt ~ post_treat | pct_wfh + pct_pub_trans +
pct_retail + pct_ent_fd| poppm2019 + share_dem, state, date,
                         data = sah, fixedeff = TRUE,
                         lambda = 0.5,
                         k = 3)
sum_ppscm4 ← summary(sah_ppscm4)
```

PPSCM Application

Plotting the new Average ATTs manually with ggplot():



PPSCM Application

Comparing to the standard event study from last lecture:

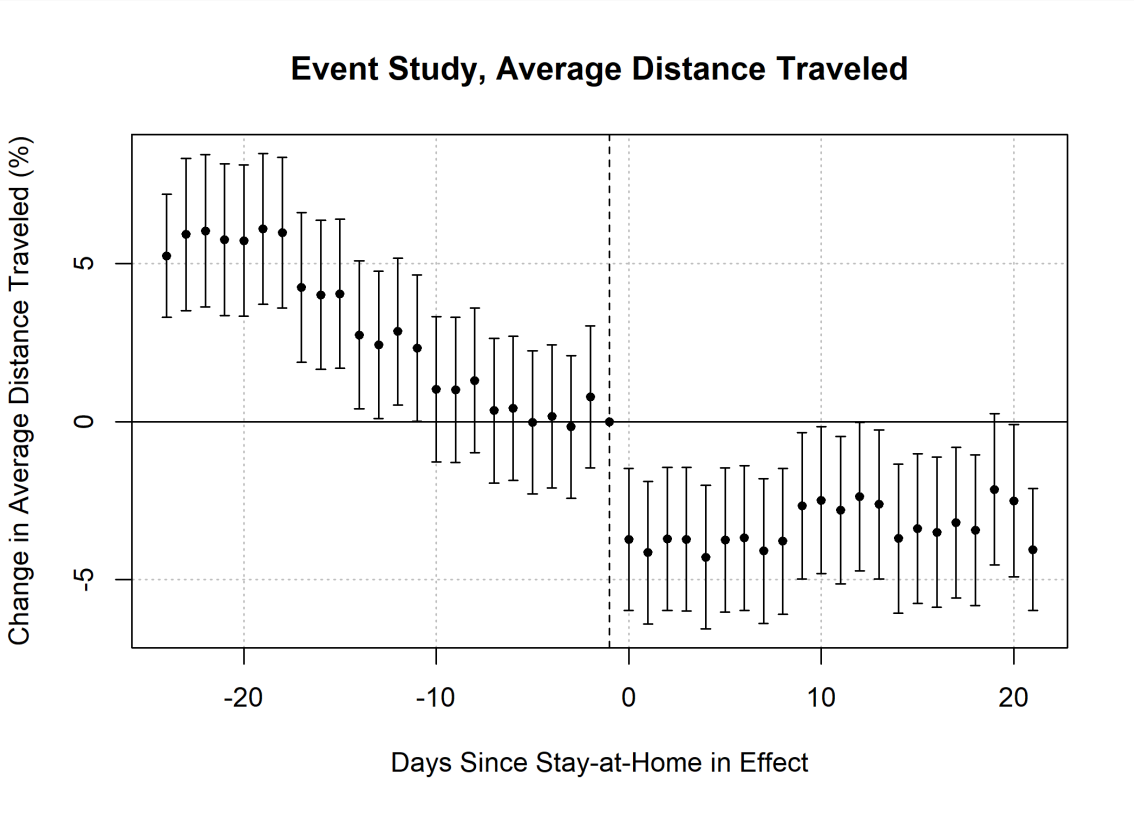


Table of Contents

1. Prologue

2. Synthetic Difference-in-Difference

3. Partially Pooled Synthetic Control

4. Synthetic Control Prediction Intervals

5. SCM Assumptions

6. Not Covered: PPSCM Application